

HyPIM: LLM Acceleration with A Hybrid ReRAM/SRAM 3D-PIM Architecture

XIAOLONG HU, College of Computer Science and Electronic Engineering, Hunan University, Changsha, China

CHUBO LIU, College of Computer Science and Electronic Engineering, Hunan University, Changsha, China and The Ministry of Education Key Laboratory of, Changsha, China

YAN DING, College of Computer Science and Electronic Engineering, Hunan University, Changsha, China

KEQIN LI, Department of Computer Science, State University of New York at New Paltz, New Paltz, United States

KENLI LI, College of Computer Science and Electronic Engineering, Hunan University, Changsha, China

Transformer-based Large Language Models (LLMs) have made significant progress in the fields of computer vision and natural language processing. The use of LLMs in embedded and real-time systems to provide more powerful intelligence capabilities is also an inevitable trend. However, the traditional von Neumann architecture encounters great challenges with transformer-based LLMs due to huge data movement overheads. Processing-in-Memory (PIM) emerges as a great potential solution by significantly closing the gap between computation and storage. Nevertheless, transformer-based LLMs usually consist of multiple transformer blocks, and the operations within each transformer block vary significantly on computation and memory access characteristics. Existing PIM designs that rely solely on SRAM, DRAM, or ReRAM cannot adapt well to transformer-based LLMs. To address these challenges, we introduce HyPIM, a 3D hybrid PIM architecture that integrates SRAM and ReRAM slices to process through their cooperation. However, HyPIM does not change the scenario where the inference latency of the attention mechanism continues to dominate the overall inference latency in the transformer-based LLMs when dealing with long sequences of data. Exploiting these insights, we then design a hardware-friendly early termination approximation algorithm and a sparse matrix multiplication strategy for the HyPIM architecture, reducing the overhead through a software and hardware co-design. Compared with the state-of-the-art Newton, TransPIM, HAIMA, HARDSEA, and H3DAtten architectures, the proposed HyPIM architecture achieves up to a $1.27\times$ – $1.67\times$ latency improvement.

CCS Concepts: • **Computer systems organization** → **Architectures**; **Neural networks**; **Real-time systems**.

Additional Key Words and Phrases: Latency, Acceleration, Large Language Models, Transformer, Processing-in-Memory, SRAM, DRAM, ReRAM, Hybrid

1 Introduction

In recent years, transformer-based LLMs have made remarkable strides in diverse areas such as computer vision [36] and natural language processing [9, 28]. There is an inevitable trend toward the use of large models in

This work was supported in part by the Programs of National Natural Science Foundation of China (Nos. 62441234, and 62532005).

Authors' Contact Information: Xiaolong Hu, College of Computer Science and Electronic Engineering, Hunan university, Changsha, Hunan, China, xlonghu@hnu.edu.cn; Chubo Liu (corresponding author), College of Computer Science and Electronic Engineering, Hunan university and The Ministry of Education Key Laboratory of "Fusion Computing of Supercomputing and Artificial Intelligence", Changsha, Hunan, China, liuchubo@hnu.edu.cn; Yan Ding, College of Computer Science and Electronic Engineering, Hunan university, Changsha, Hunan, China, ding@hnu.edu.cn; Keqin Li, Department of Computer Science, State University of New York, New Paltz, New York, USA, lik@newpaltz.edu; Kenli Li, College of Computer Science and Electronic Engineering, Hunan university, Changsha, Hunan, China, likl@hnu.edu.cn.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 1558-3465/2026/6-ART

<https://doi.org/10.1145/3820366>

embedded and real-time systems [32, 50]. The application of these models in such systems is continuously expanding and deepening. By providing more powerful intelligence capabilities, these systems can become more intelligent, efficient, and responsive, thereby serving a wide range of practical application scenarios. Furthermore, in real-time inference applications involving intelligent LLMs and other large models on the embedded side, the most important requirement is low latency. Specifically, the goal is to accelerate models under resource constraints [18, 20] and improve the real-time response time of these models [3, 33, 51], such as LLMs, in practical applications.

However, the performance of transformer-based LLMs often benefits from a massive growth in the number of model parameters and computational costs. In traditional von Neumann architectures, the processor and memory are separated. The transformer's frequent matrix multiplication operations necessitate constant data transfers between these two components, resulting in considerable latency. The data movement time of transformer-based LLMs on the traditional von Neumann architecture exceeds 60% [54]. Thus, data movement has emerged as a non-negligible bottleneck of transformer-based LLMs.

Processing-in-Memory (PIM) architectures [2] enable computation to be performed directly within memory, thereby significantly reducing data-movement overhead, and have been regarded as a promising approach to accelerate transformer-based LLMs. For example, TransPIM [54] using bit-serial row parallel PIM operations substantially reduce data-movement overhead, which highlights the potential of PIM for LLM acceleration.

To date, PIM architectures have been widely used to accelerate convolutional neural networks (CNNs). Compared with CNNs that exhibit relatively regular computation patterns, transformer-based models show pronounced heterogeneity across modules in both computation and memory-access characteristics. Therefore, although recent efforts have begun to explore PIM-based accelerators [54] for transformer-based models, such module-level disparities make it difficult for existing PIM designs to execute all LLM modules efficiently.

Firstly, current architectures often overlook the variability of access and computational characteristics among transformer modules, which means they cannot adapt well to PIM architectures based on sole hardware design. Dynamic Random Access Memory (DRAM), Static Random Access Memory (SRAM), and Resistive Random Access Memory (ReRAM) are all hot topics in the research of PIM architecture. However, they differ in structure, operational principle and performance, so PIM architectures based on different hardware designs are often used to handle different types of tasks. For example, area efficiency of SRAM is low, but read/write efficiency and durability are very high, so SRAM-based architectures are more suitable for dealing with small-scale applications that require fast and frequent access, such as dot product in the attention module. Area and read/write efficiency of ReRAM is high, but its endurance is lower. Thus, ReRAM-based architectures are more suitable for applications that require high-capacity storage and a certain level of efficiency, such as linear layers in the transformer. Therefore, the efficient processing of the complex structure of transformer-based LLMs requires the collaboration of multiple hardware PIMs.

Secondly, the traditional interconnection of multiple hardware is a separated connection architecture based on the external bus and interface, which leads to significant penalties in latency, bandwidth, and storage efficiency. Simultaneously, the increase of chip computing power requires more computing units, which leads to significant losses in energy and area efficiency [23]. Therefore, PIM architectures requires more efficient on-chip network and higher storage density.

Finally, most existing PIM architectures are aimed at computation-intensive tasks, such as CNNs, which use all resources to process the tasks of one layer at a time. This strategy leads to large and frequent data transfers when processing memory-intensive tasks. Therefore, it is necessary to plan the task execution flow and dataflow of LLMs reasonably. Additionally, the computational complexity of the attention mechanism is proportional to the square of the sequence length. As the sequence length increases, the latency of attention will dominate the overall latency. Unfortunately, the attention optimization strategy used in traditional von Neumann architectures is not applicable in PIM architectures due to the complex processing and mapping of data.

Based on the above discussion, we introduce a Hybrid 3D PIM architecture (HyPIM) that improves parallel efficiency and decreases the inference latency of transformer-based LLMs. To address variations in resource demand among transformer modules, we employ cooperative SRAM and ReRAM arrays for efficient computation for each type of task in the model. Additionally, in order to improve the memory utilization and efficiency within limited chip space, we utilize monolithic 3D (M3D) integration techniques [22] for stack multiple memory dies. Furthermore, we also design an optimization strategy for the attention mechanism applicable to the PIM architecture, which greatly reduces the computational latency. The contributions of this work are as follows:

- A hybrid 3D PIM architecture, called HyPIM, is proposed. Stacked ReRAM-based memory slices are utilized for tasks that demand extensive storage capacity, including linear operations. Meanwhile, stacked SRAM-based memory slices are employed to execute the dot product and matrix multiplication operations in the attention module, which need high efficiency.
- A mixed-granularity parallel acceleration strategy for transformers is designed. The execution order and dataflow pipeline of the transformer modules in the architecture is planned rationally.
- An attention acceleration strategy is implemented on the HyPIM architecture. The use of the computational properties of softmax allows for the early termination of computations that have no impact on the results. In addition, the data-aware mapping strategy enhances the efficiency of sparse matrix multiplication operations.

Compared to the state-of-the-art Newton, TransPIM, HAIMA, HARDSEA, and H3DAtten architectures, our proposed HyPIM achieves up to $1.27\times$ – $1.67\times$ latency improvement, while also demonstrating strong robustness and scalability.

2 Background and Motivation

2.1 Background

As shown in the Figure 1, there are three common architectures for large language models: encoder-only models (e.g., BERT), decoder-only models (e.g., GPT), and encoder-decoder models (e.g., BART). Encoder-only models and decoder-only models only perform a standard layer-by-layer forward pass, while the decoder of encoder-decoder models needs to receive the output of the encoder at the same time, which is used to generate more relevant results. In general, encoder-only is suitable for comprehension type tasks, decoder-only is suitable for generation type tasks, and encoder-decoder is suitable for tasks with strongly correlated inputs and outputs, such as translation. This paper describes a common structure of an encoder/decoder block, whose main modules are illustrated in Figure 1:

Multi-head Attention Module:

$$Q = XW^Q, \quad (1)$$

$$K = XW^K, \quad (2)$$

$$V = XW^V, \quad (3)$$

where X is a positionally embedded input matrix representing a sequence with N tokens and M hidden dimensions, W^Q , W^K , W^V are three weight matrices with $M \times M$ dimensions in fully-connected layer (FL), which is a linear layer, to generate the Query(Q), Key(K) and Value(V) matrices with $N \times M$ dimensions. Then

$$S' = QK^\top, \quad (4)$$

$$S = \text{Softmax}(S' / \sqrt{M}), \quad (5)$$

$$\mathcal{A} = SV, \quad (6)$$

where the $\text{Score}(\mathcal{S})$ matrix with $N \times N$ represents the similarity between all elements, $\mathcal{S}'$ is the initial calculated score matrix, whose dimensions are the same as $\mathcal{S}$. After division and softmax operation, the final $\mathcal{S}$ can be obtained. And $\mathcal{A}$ is a matrix with $N \times M$ dimensions.

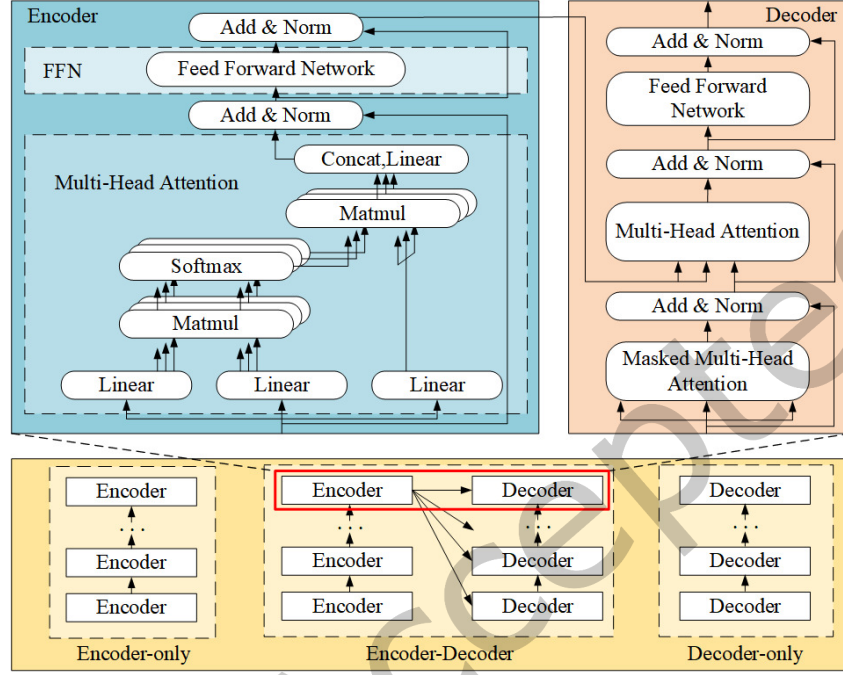


Fig. 1. Structure of several general transformer-based LLMs.

When we add the **multi-head mechanism**, all of the above matrices become sub-matrices partitioned by H heads, e.g., $Q_h, K_h, V_h, W_h^Q, W_h^K, W_h^V, S_h, A_h$. $h \in (0, H - 1)$. And $\mathcal{A} = \text{concat}(\mathcal{A}_0, \mathcal{A}_1, \dots, \mathcal{A}_{H-1})$, then $\mathcal{Z} = \mathcal{W}^Z \mathcal{A}$ and an Add&Norm layer, where $\mathcal{Z}$ is the output matrix of the attention module, $\mathcal{W}^Z$ is a weight matrix in the last FL layer.

In the decoder block, **Masked Multi-Head Attention** is used. The mask is primarily employed to prevent the model from attending to future positions. The masking matrix is typically an upper triangular matrix containing negative infinity values (or very small values), ensuring that the softmax operation ignores the weights at these positions. For example, unmasked locations are usually assigned a value of 0, while masked locations are assigned negative infinity. To compute the attention matrix, the masking matrix is added to the original attention score matrix.

$$\mathcal{S}'_{\text{masked}} = \text{mask} + \mathcal{S}', \quad (7)$$

where the *mask* refers to the masking matrix, and $\mathcal{S}'_{\text{masked}}$ is the score matrix with the mask applied, both having the same dimensions as $\mathcal{S}'$. The rest of the computation and attention modules remain essentially the same.

After the attention module, an Add&Norm layer is immediately followed, which belongs to the residual structure. This layer normalizes the result of adding the input and output of the attention module, which can enhance the expressiveness of the output results.

Feed-Forward Neural Network (FFN) Module:

$$\mathcal{O} = \text{FL}(\text{FL}(\mathcal{Z})), \quad (8)$$

the Output (O) matrix of the FFN module with $N \times D$ dimensions is obtained from two FLs. The final output of the transformer also needs to go through an Add&Norm layer that normalizes the result of adding the input and output of the FFN module.

Table 1. Device characteristics of memory technologies

	Mainstream memories		Emerging memories
	SRAM	DRAM	ReRAM
Cell area	$> 100F^2$	$6F^2$	$4 - 12F^2$
R/W time	$\sim 1ns$	$\sim 10ns$	$< 10ns$
Endurance	$> 10^{16}$	$> 10^{16}$	$> 10^6 - 10^{12}$
Write energy	$\sim fJ$	$\sim 100fJ$	$\sim 0.1pJ$

2.2 Motivation for Accelerate Transformer

Frequent matrix multiplications during the inference process of transformers result in increased data movement costs, which dominate the inference latency [54]. Thus, PIM architectures minimizing data movement have become a popular approach for accelerating neural networks. While PIM architectures for CNNs have been extensively researched, they do not directly apply to transformers. The following challenges exist:

Challenge 1: The singular storage hardware PIM architecture is unsuitable for the structure of the transformer model.

Existing PIM architectures are often built on a single storage hardware [19, 44]. This design fails to consider the varying storage and computational resource demands of different network modules, resulting in poor resource utilization and compute efficiency. As is shown in Table 1, the characteristics of memory device technologies differ significantly [25].

The weights of the pre-trained model remain unchanged during the inference process, and they can be pre-loaded into memory to avoid repeated read/write operations. However, this approach consumes a significant amount of memory resources. In contrast, all data of the attention mechanism are generated and loaded into the computational array during the inference, thus requiring high read/write operation frequency and efficiency.

The computational delay caused by the mismatch between task and hardware characteristics can significantly impact the overall inference delay. Therefore, compared with the use of DRAM-PIM and traditional accelerators to accelerate transformer inference [29, 38], hybrid PIM architectures based on ReRAM with high memory density and SRAM with high read/write efficiency are ideal for accelerating complex transformers.

Challenge 2: The traditional connection between the two types of hardware incurs significant penalties in bandwidth, storage, and computing efficiency.

The traditional connection mode between hardware usually involves using external buses or interface data transfer channels to connect different hardware supporting PIM chips and processors responsible for processing tasks. This segregated architecture incurs severe losses in bandwidth, storage, and computational efficiency, especially when dealing with large-scale data or real-time applications. In the traditional 2D architecture, multiple different storage devices and processors are usually located in different packages, which leads to a waste of storage space and area, and increases the data transmission path, thus affecting the computing efficiency [23, 34].

To overcome these limitations, a hybrid 3D PIM architecture based on 3D memory structure is proposed. The architecture stacks multiple ReRAM and SRAM dies through M3D integration technology to eliminate the inherent limitations of the traditional connection mode. In this way, the physical transmission distance between the storage unit and the computing unit is greatly shortened, and the data exchange delay between the slices is significantly reduced. At the same time, due to the increase of vertical interconnection density, the bandwidth is significantly enhanced, which can not only improve energy efficiency during cross-layer data transmission,

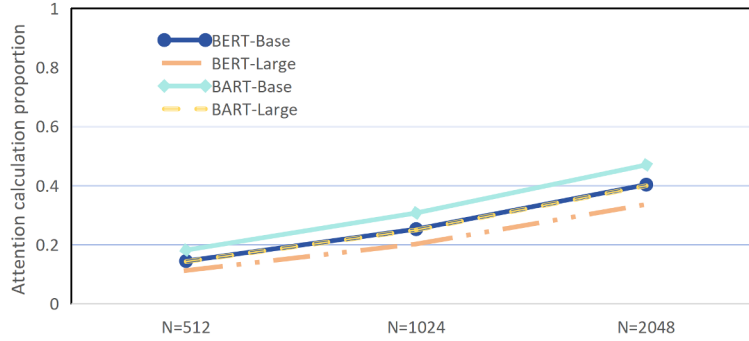


Fig. 2. Attention calculation proportion of transformer-based LLMs (N : number of tokens)

but also alleviate the power consumption and performance bottlenecks caused by the long interconnection in traditional two-dimensional interconnection architectures [4, 45, 49]. Each chip is divided into multiple partitions and uses Monolithic Intertier Vias (MIVs) to connect with other chips. Each partition operates independently, and data transmission for each partition is realized through MIVs and on-chip communication paths, thereby reducing the loss of computational efficiency.

Challenge 3: Currently available attention optimizations are difficult to map efficiently onto PIM architecture.

The attention mechanism calculates similarity among all elements with a time complexity of $O(N^2)$. As shown in the Figure 2, processing lengthy sequence data imposes severe computational pressure on the attention mechanism, resulting in significant performance costs for the model. In another sense, the model's size is restricted.

Various optimization algorithms have been designed for attention mechanisms, including techniques such as pruning, quantization, and approximate computation. Quantization and approximate computing usually rely on general-purpose computing units to perform complex format conversion and mapping of data. Then, some works also use the cache and memory access optimization in the von Neumann architecture to improve computational efficiency. However, in PIM architecture, the computing unit is limited to the native operations of the memory array, and its computation and storage are highly coupled, which makes it difficult to efficiently support complex indexing, data rearrangement and dynamic control flow. These optimization methods are not only difficult to map on PIM, but also may introduce additional data movement and energy consumption overhead, which weakens the energy efficiency advantage of PIM.

In this paper, we focus on the two primary computations that have the most significant effect on the attention mechanism: the S dot product calculation and the A matrix calculation. Combining their different computing characteristics and hardware features, a software and hardware collaborative strategy suitable for the PIM architecture was designed to improve efficiency.

3 HyPIM overview

HyPIM is a PIM accelerator specifically designed for transformer inference. Its architecture is built on M3D Integrated Circuit (IC), a currently promising 3D IC technology [4, 45, 49] that stacks multiple functional layers capable of storing and computing. The highly integrated and heterogeneous architecture is compatible with CPUs, GPUs, and other devices. This section provides a comprehensive overview of the dataflow, fundamental components and parallel strategy of HyPIM. Subsequent sections introduce the optimization strategies for the attention mechanism.

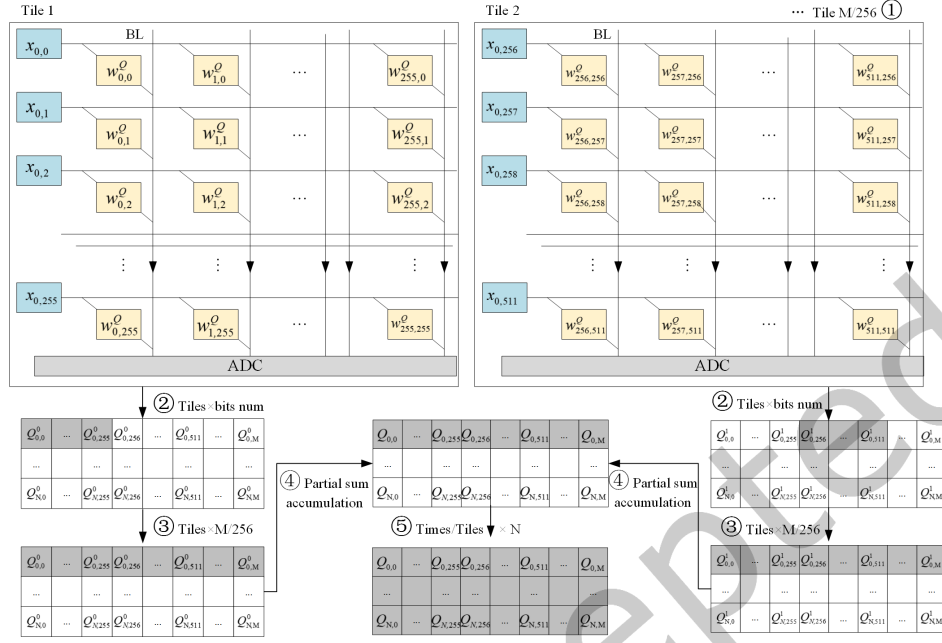
3.1 Execution flow of Transformer modules on HyPIM

ReRAM-PIM has high memory density and good read/write efficiency, which is suitable for processing tasks with large data scale. We can make full use of all the storage resources to maximize the parallelism to complete the task. For example, weight data can be copied into more than one computing array, and the high degree of parallelism between tokens allows tokens to be calculated separately in different arrays. SRAM-PIM has high processing efficiency and high read and write durability, so it is more suitable for processing tasks that require high read/write frequency and efficiency. At this point we can divide the tasks according to the degree of parallelism within the tensor. For example, in the multi-head mechanism, the tensor is divided into blocks according to the head and loaded into different arrays for calculation. In this chapter, we divided the main computation of the transformer into four parts, the first is QK^T linear layer, then the Dot product $S' = QK^T$, and the Matrix multiplication $\mathcal{A} = S'V$ in attention, and finally the multiple linear layer computations including FFN. Based on the above discussion, ReRAM-PIM is more suitable to deal with linear operation in attention and FFN module, while SRAM-PIM is more suitable to deal with other operations in attention module. The array size is set to 256×256 . The execution process of transformer modules on HyPIM is shown in Figure 5 and will be described in detail in Section 3.3.

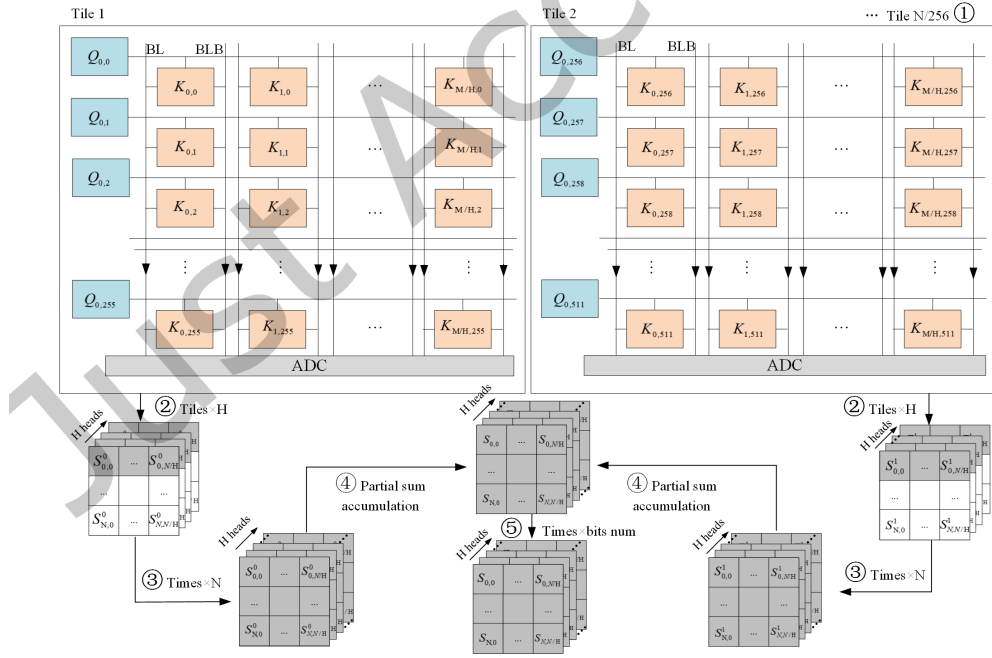
Dataflow in ReRAM-PIM: As mentioned above, the linear operation such as QK^T linear layer and linear layer in FFN is more suitable for execution on ReRAM-PIM. As shown in Figure 3(a), we take the calculation process of Q as an example to describe the calculation process on ReRAM-PIM. In the pre-processing process before the calculation, the weight data of each layer, W^Q , W^K , W^V , etc. are loaded into the array in advance. The calculation does not need to consider the parallelism between the multi-head mechanism, and can be directly divided according to the size of the array. First, W^Q can be divided into $M/256$ sub-matrices W_i^Q by column, where $i \in (0, M/256 - 1)$. Then, W_i^Q can also be divided into $M/256$ submatrices by row, and these submatrices are loaded into the ReRAM array. ① A single token vector x_0 will also be divided into $M/256$ sub-vectors $[x_{0,0}, \dots, x_{0,i}, \dots, x_{0,255}]$, and each element in the sub-vector will be calculated with each row in the array and accumulated to obtain the partial sum data. $M/256$ tiles will be able to complete all the partial sum calculation of some elements in the Q_0 vector. ② Since ReRAM can only store 1 bit per cell, it needs bit_num (n) arrays to calculate the complete weight at the same time. ③ The computation to obtain all the sub-vectors of Q_0 requires an additional $M/256 \times$ array, that is, a total of $M/256 \times M/256 \times bit_num$ arrays to obtain all the partial sums of Q_0 . ①②③ can be done simultaneously. ④ After completing the accumulation of all obtained partial sums, all computations of Q_0 are completed. If there are enough arrays, the weights of this layer can be copied to other arrays so that multiple tokens can be computed at the same time (tiles $\times N$), and if the storage is full, other tokens can be computed in series.

The above computational flow takes full advantage of the parallelism between computing resources and tokens. Tokens can be loaded into each array in a pipelined parallel mode, so the performance of HyPIM is not affected even if the number of tokens is large. Q , K , and V are sent to SRAM-PIM through the high-speed memory bandwidth of HyPIM architecture after being calculated on ReRAM-PIM to participate in other calculations in attention. The same holds for the rest of the Linear Operations, which are suitable for execution on ReRAM-PIM, including FFN and the last layer of the attention module. Firstly, the weight matrix is divided into $M/256$ submatrices by column, and then divided into $M/256$ submatrices by row, loaded into ReRAM array, and then the same calculation process is adopted.

Dataflow on SRAM-PIM: Suitable operations to perform on SRAM-PIM mainly include dot product $S' = QK^T$ and matrix multiplication $\mathcal{A} = S'V$ in attention. As shown in Figure 3(b), we take the computation process of S as an example to describe the computation process on SRAM-PIM. The Q and K matrices required for calculation are calculated by the linear layer and transmitted to the SRAM array. The calculation of QK^T starts at the same time, that is, the calculation results of QK can be obtained in parallel. Then, K is preferentially loaded into the



(a) Dataflow in ReRAM-PIM



(b) Dataflow in SRAM-PIM

Fig. 3. Dataflow in HyPIM

SRAM array as a weight matrix. At this time, $\mathcal{K}$ also needs to be divided into multiple arrays according to the number of bits, but because the complete data of each bit needs to be obtained step by step in the subsequent optimization strategy, this step is moved to the end. The computation of $\mathcal{S}$ needs to consider the multi-head mechanism. Firstly, the transpose $\mathcal{K}$ is divided into H submatrices by row, and then into $N/256$ submatrices by column, and these submatrices are evenly divided into each SRAM array. Therefore, a total of $H \times N/256$ arrays are needed to ensure that all results of $\mathcal{S}_0$ vector can be obtained in parallel. ③ Each row vector of $\mathcal{S}$ is then computed separately, again due to the finite array, here multiple time steps are chosen to complete the computation of different vectors. ④ The calculation of each head does not affect each other, so that the calculation results of each head are added to the corresponding partial sum. ⑤ Finally, the final result is accumulated bit by bit in different time steps.

After the accumulation, scaling is needed, and then the softmax calculation is completed by the lookup table, and the resulting final $\mathcal{S}$ matrix is stored in the buffer of the SRAM-PIM for the calculation of the next part of matrix multiplication $\mathcal{A} = \mathcal{S}\mathcal{V}$. The computation of $\mathcal{A} = \mathcal{S}\mathcal{V}$ is also more suitable to be performed on SRAM-PIM. $\mathcal{V}$ has been computed by ReRAM-PIM and sent to the SRAM chip along with $Q\mathcal{K}$ through the memory bandwidth. Similarly, $\mathcal{V}$ is divided into H submatrices by column, and then the submatrices are divided into $N/256$ submatrices by row, which are loaded into the SRAM array respectively. Each row vector in $\mathcal{S}$ can then be divided into $N/256$ sub-vectors by column to participate in the calculation, and each row vector can be calculated separately. The accumulated output results need to be transmitted to ReRAM-PIM through inter-layer interconnection to complete the calculation of other linear layers.

3.2 Infrastructure of HyPIM

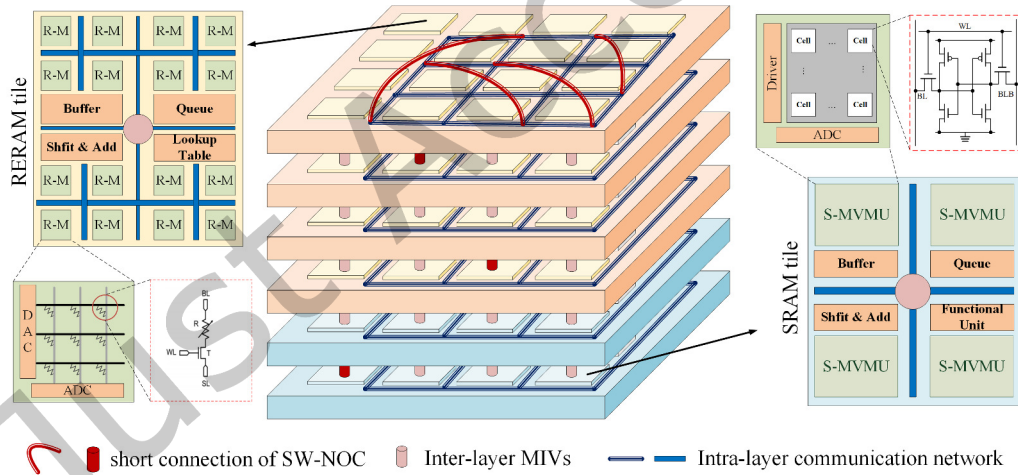


Fig. 4. Overview of HyPIM Architecture

Figure 4 provides an overview of the HyPIM architecture, which supports the execution flow described in the previous section. The key components are described below.

Matrix-Vector Multiplication Unit (MVMU). The MVMUs of HyPIM are classified into two types: the ReRAM-based MVM unit (R-MVMU); and the SRAM-based MVM unit (S-MVMU). R-MVMU has the capability to pre-load the weight data into the computational array prior to inference, unlike S-MVMU which needs to gradually load the data into the computational array during the inference process. The array's original size is set

to 256×256 , which is well suited for long-sequence workloads. All rows share the input buffer of each MVMU, and all columns share the output buffer. In each cycle, the controller of the MVMU will read the input data required for each row from the input buffer and load them into the input registers of the corresponding rows. The controller will records the indices to route partial sums to the corresponding accumulation. The MVMU then converts the digital signals into analog voltages applied to each row of the MVMU via DACs and the array starts performing analog matrix multiplication operations. The difference is that the output data of the R-MVMU is represented as the accumulated current value, while the output data of the S-MVMU is represented as the accumulated voltage difference between bitline (BL) and bitline bar (BLB). Eventually, they are both converted back to digital signal values via ADCs for transmission to the next module.

Accumulate Queue. The MVMU arrays in this architecture only store 1-bit values per cell, and the n -bits precision values are assigned to different arrays to complete the calculation, and the calculation results of each array go through the S&A unit to obtain the actual data and accumulate to obtain the final n -bits calculation results. The data obtained by the MVMU are placed in the corresponding accumulation queue according to the corresponding index. In R-MVMU, all data are processed in parallel and the resulting computations are saved to the corresponding accumulation queue, completing partial sum accumulative processes. In contrast, S-MVMU utilizes a time-domain SIMD-based design due to resource limitations. This design can be selected to execute multiple MVM operations serially in multiple cycles. The results of each computation cycle are stored in the accumulation queue, based on their indices, to complete the overall accumulation process. Once the accumulation queue connected to a specific index has completed all the accumulations, the data within the queue are extracted and deposited into the output buffer.

Nonlinear and Functional Units. Nonlinear calculations are performed through lookup tables, stored in a ReRAM slice with high storage density, providing a means of high accuracy computation. To reduce the size of the nonlinear calculation lookup table, we split the n -bits data into high and low $\frac{n}{2}$ -bits. This method allows us to lookup data in the table based on the high and low $\frac{n}{2}$ -bits, and integrate them to obtain the final result. The functional unit mainly includes the division unit and the addition unit. The division unit is mainly on SRAM slice. After the element of S' is calculated by dot product, the corresponding element is directly obtained through the division unit, so that the corresponding element of S can be quickly found in the lookup table. The addition unit adds the input elements and output elements of the attention module or FFN module after the output elements are obtained, so that the next normalization process can be completed quickly.

Communication Method. The functional layer employs a communication method based on M3D-based 3D small-world network-on-chip [8] (M3D-SW-NoC) which is designed with a topology that possesses a short average path length and high global connectivity. Improved local connectivity of nodes is achieved with the addition of random short-distance connections while maintaining global connectivity between further nodes. This design results in shorter communication paths between nodes, thereby reducing communication delays and enhancing local communication efficiency. The intra-layer communication technique combines a 2D mesh topology with a tree topology. The single chip array is composed of several uniformly situated tiles, which are connected via a 2D mesh communication network as a component of the M3D-SW-NoC. The tree topology is implemented within the tile to link each S-MVMU/R-MVMU arrays and other functional modules.

3.3 Mixed Granularity Parallel Strategy

As mentioned in Section 2, HyPIM maps S and $\mathcal{A}$ calculation to SRAM-PIM, while offloading linear layers (e.g., Q/K/V calculation and FFN) to high-capacity ReRAM-PIM. Figure 5(a) depicts the coarse-grained execution timeline of Transformer modules on HyPIM. Such an execution process can not only realize the overlap of communication and computation, reduce some unnecessary waiting delays, but also solve the problem of low

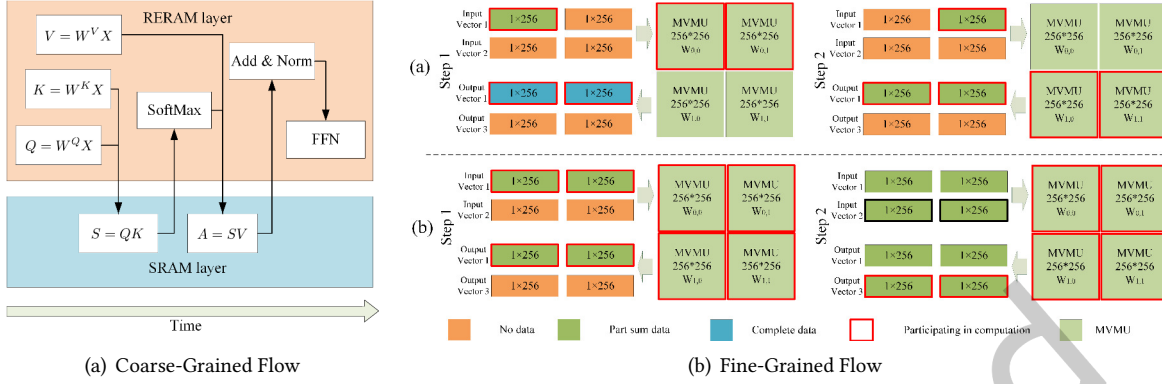


Fig. 5. Mixed Granularity Parallel Strategy

resource utilization and high latency that may be caused when different types of tasks are executed on architecture based on different hardware.

As shown in Figure 5(a), in the calculation process, the calculation of Q , K , and V can be started in parallel on ReRAM-PIM, and the results are transmitted to SRAM-PIM through high-bandwidth inter-layer interconnection. Among them, the calculation result of Q , K will be used as the input of S , so the calculation of S must be started after the calculation of Q , K is completed. Then, the calculation results of V and S are jointly used to generate A , so the start time of the calculation of A needs to be later than the completion time of the calculation of S and V . Both S and A are computed on SRAM-PIM, where Softmax is implemented using lookup tables (LUTs) stored in ReRAM. Subsequently, the result of A will be transmitted to ReRAM-PIM for subsequent computation. Similarly, the computation of the FFN depends on the output of A , so its start time must be later than the completion time of A 's computation.

Most traditional architectures execute only a single network layer at a time, which means that all compute units are used only for that layer, thus achieving significant parallelism. However, this is not applicable in PIM architectures with high storage density and may lead to a considerable waste of communication and storage resources. Therefore, it is imperative to develop an appropriate fine-grained pipeline for PIM architectures. If the computation of the previous layer generates enough output data to support the computation of part of the array in the later layer, our architecture will forward this data to the array containing the parameters of the next layer through the NoC, thus starting the computation of the subsequent layers ahead.

In Figure 5(b), the weight matrix of size 512×512 is divided into four parts, which are stored in four MVMUs. As shown in part (a), in step1, when a subvector of input vector 1 arrives, it can perform basic operations with $W_{0,0}$ and $W_{0,1}$ stored in MVMU, and HyPIM immediately starts to calculate and generate the partial sum of output vector 1. Subsequently, in step2, when the remaining subvectors of input vector 1 arrive, the corresponding computation continues to obtain the remaining partial sum, which is accumulated with the previous result to obtain the complete output. As shown in part (b), when a complete input vector is ready, the weight matrix will participate in the calculation at the same time to obtain the complete result of the output vector. In step 1, the first input vector 1 is calculated to obtain the output vector 1. In step 2, the newly arrived input vector 2 is calculated to obtain the output vector 2.

Using the above strategy, we can realize the pipelining illustrated in the Figure 6. As shown in Figure 6(a), taking QKV to $S' = QK^T$ as an example: once the QKV task in the ReRAM layer has loaded a portion of data that is sufficient to start computation (the time marked by the first red solid line), execution can be launched immediately. After the first batch of output data is produced (the time marked by the first red dashed line), the

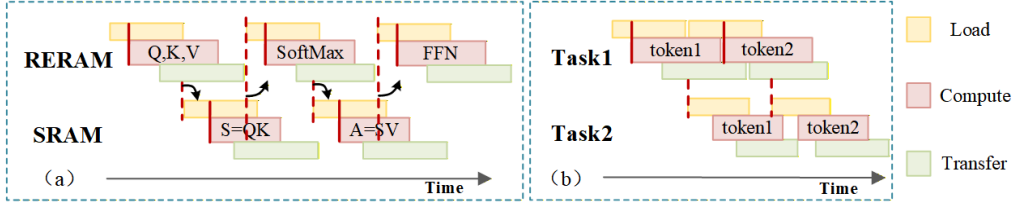


Fig. 6. Mixed Granularity Parallel Pipeline

results can be delivered to the SRAM layer that executes $S' = QK^T$ via intra-layer transfer. The SRAM layer then starts receiving and buffering these intermediate results; when the buffered data accumulates to the amount required for a tiled computation in that layer, the $S' = QK^T$ task begins execution at the time marked by the second red solid line. Subsequent tasks proceed in the same pipelined manner.

Figure 6(b) further shows the execution of the same task across multiple tokens. While task1 is loading all data for token1, it already starts partial computation, and once the computation finishes, it transfers the results to the array running task2. Meanwhile, the buffer space in task1's array that is used for token1 can be partially released and reused to prefetch data for token2. Through this “compute-while-transfer” and “load-while-compute” scheme, efficient overlap between intra-layer communication and computation is achieved.

4 Attention mechanism synergy optimisation strategy

The attention module is a crucial component of the transformer model. When dealing with a long sequence of data, its computational characteristics result in the attention module taking up the majority of the computational and latency costs of the overall inference process. This makes the attention module a performance bottleneck for transformer-based LLM inference. Hence, optimizing and accelerating the attention component is imperative.

However, current approaches for expediting the attention module are often inapplicable in PIM architectures. The existing attention optimization on PIM mostly adopts static threshold or top-K strategy [34], which often brings accuracy loss while reducing redundant calculations. Our proposed dynamic threshold mechanism can adaptively retain critical non-zero values and maintain higher accuracy while reducing redundancy. In addition, although HARDSEA [34] and other methods can skip zero values, the array utilization is low. Our hardware-software collaboration strategy can balance the efficiency and resource utilization in sparse computing. This chapter seeks to examine acceleration techniques for the attention mechanism in PIM architecture, with the objective of achieving more effective attention acceleration. We focus on two core computational operations in the attention mechanism.

4.1 Early Calculation Termination Strategy

The first core operation of the attention module is $S' = QK^T$, which involves performing dense matrix multiplication whose outputs become effectively sparse after softmax. The attention module employs softmax to selectively attend to elements in the input sequence that hold relevant information by assigning higher attention weight to them. This method accentuates the disparities between a restricted number of large elements and the majority of smaller ones, consequently producing a substantial quantity of negligible data that does not influence the outcome. If we can establish that the data resulting from the primary calculations is small enough, we can halt the calculations prematurely.

The A^3 [15] approximation algorithm is also an early termination algorithm based on this observation. The algorithm sorts the elements of each column in the K matrix in the preprocessing process, so that the results obtained in the calculation process can gradually converge to the final calculation result. When the difference between the partial sum of a stage and the maximum value is too large, it can be considered that the difference

between its final result and the maximum value is still too large. Therefore, the subsequent calculation process of this partial sum can be discarded. This kind of approximation algorithm can greatly reduce the amount of computation in the execution process by setting the appropriate threshold value, and does not affect the accuracy. Unfortunately, in PIM operations that require a high degree of parallelism, it may lead to repeated loading of weight data and aggravate resource loss.

Each unit of SRAM-PIM analog circuit array only supports 1 bit of data, so under the limited computing resources, the bit-serial computing can be used to calculate the operation result of each element bit by bit from high to low, and finally accumulate to get the final data. In the calculation process in SRAM-PIM, the weight data is first loaded into the SRAM array. In order to ensure a higher degree of parallelism, different bits of weight data are loaded into different arrays. Each array will perform a full calculation of each bit weight. The input data is then loaded bit-by-bit into the array, processed through multiple stages, from the highest to the lowest bits, and summed up to obtain the final result.

The difference in the partial sum of multiple high bits (such as R) often represents the difference in the final result. Therefore, a judgment mechanism can be added after the first R bits of calculation to determine whether the operation of some elements can be terminated early. If the difference between the partial sum and the maximum value is too large, the subsequent calculation will be abandoned, otherwise, the subsequent calculation of the partial sum will continue. In the subsequent calculation, the judgment mechanism is continuously added until the final result is obtained.

An important part of the design of implementing an early termination strategy is how to determine which computations to discard. Most of the existing judgment strategies are based on static methods, where a fixed

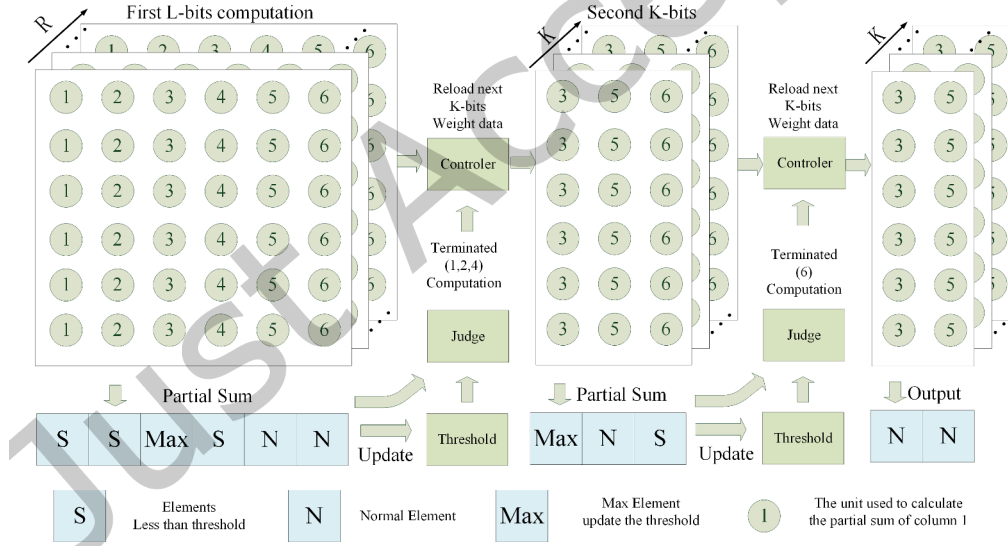


Fig. 7. Co-design for Early Calculation Termination Strategy

threshold is set in advance to select the appropriate elements, or a fixed number of high-scoring elements are selected. However, in the case of poor data distribution, these static judgment methods may eliminate some high-scoring elements, affecting the final result accuracy, or may leave some low-scoring elements, resulting in little efficiency improvement. Therefore, the dynamic threshold can be chosen to decide which elements to keep in the calculation. Similar to the dynamic threshold setting scheme in A^3 , the threshold will be determined

by the maximum value t_{max} of the partial sum within the same phase element, set as $T = 100(1/e^{t_{max}(1-\alpha)})$, where $\alpha \in (0, 1)$ is an adjustable value, representing a more aggressive or conservative strategy. Here, the initial threshold is set to $t_{max}(1 - \alpha)$, which means that only the elements of the obtained score vector that differ from the maximum value by less than α times are retained. Since the subsequent Score needs to go through the softmax operation, this threshold can be further mapped to T by the formula, that is, only the value of the first $T\%$ is retained in the score matrix after softmax. In this way, the threshold can be adjusted flexibly according to the specific scene, so as to realize a more adaptable dynamic pruning strategy.

The above scheme achieves early termination on the SRAM array, but still leads to a lot of idle computing resources. To implement the approximation scheme described above, the array needs to be replanned. Multiple SRAM arrays of different sizes (e.g., 256×256 , 256×128 , 256×64 , ...) are provided in the HyPIM, in the stage of the first R bit operation, using 256×256 to load all the elements as much as possible and the weight data needed in the calculation. After the calculation of some elements is discarded by the judgment mechanism, if the number of existing elements is less than $256/128/64$, the $256 \times 256/128/64$ array can be selected to load only the weight data of the required elements for subsequent calculation. As shown in the Figure 7, after the calculation of R bits, the elements of row index(1,2,4) are determined to require no further computation, and the smaller array is selected to load weight data of row index(3,5,6). In the subsequent calculation, the element of row index 6 is determined to require no further computation, and the smaller array is selected to load the required weight data of row index(3,5). Early termination of idle computing resources can be used for other computing tasks to improve the efficiency of the pipeline, or increase parallelism to improve the overall reasoning efficiency.

After HyPIM adopts the above settings, it is able to flexibly select the appropriate array for loading and computing according to the actual number of elements needed in each computation stage. In this way, when a partial row index is judged to be no longer necessary to continue the computation, it can be switched to a smaller array, avoiding the long idle time of invalid cells in a large array. At the same time, small arrays can be assigned to other computing tasks when they are not in use, so as to ensure that all arrays can be effectively used. It can be seen that this strategy not only reduces the idle time of array cells, but also makes full use of hardware resources through flexible scheduling and resource reuse.

The decision mechanism incurs an $O(N)$ cost per invocation. Therefore, the additional decision overhead is $O(N) \times (\text{number of decisions})$, which is typically a small constant-factor cost relative to the dominant computation term, i.e., $O(N^2M) \times (\text{number of bits})$.

4.2 Co-Optimization Strategy for Sparse Matrix Multiplication

The second core operation of the attention module is $\mathcal{A} = \mathcal{S}\mathcal{V}$, which involves sparse matrix multiplication after the above strategy. Specifically, the large number of zero elements in the sparse matrix leads to waste of computational resources and increased inference latency. Most existing work stores and processes sparse data using formats such as COO or CSR. However, the hardware architectures of existing accelerators (e.g., GPUs) are inefficient for sparse operations (e.g., SpMV/SpMM). Hardware devices based on systolic array architecture, such as TPU, do not support to skip zero calculation due to the strict internal connectivity. PIM architecture has rigid interconnection similar to systolic array, and the processing of sparse matrix multiplication will lead to a lot of waste of resources.

However, in the calculation of SRAM-PIM, the dense matrix $\mathcal{V}$ of the module is pre-loaded into the SRAM array as a weight. Since the calculation process of each score vector in the score matrix basically does not affect each other, each array is actually responsible for the calculation of the $\mathcal{V}$ matrix and the sparse row vector in the matrix $\mathcal{S}$. Therefore, the calculation of this part of the attention mechanism can be abstracted into the calculation between multiple sparse vectors and dense matrix. Each row of compute units in the array is responsible for the calculation of a single input element with multiple weights stored in that row. Therefore, in the case of no complex

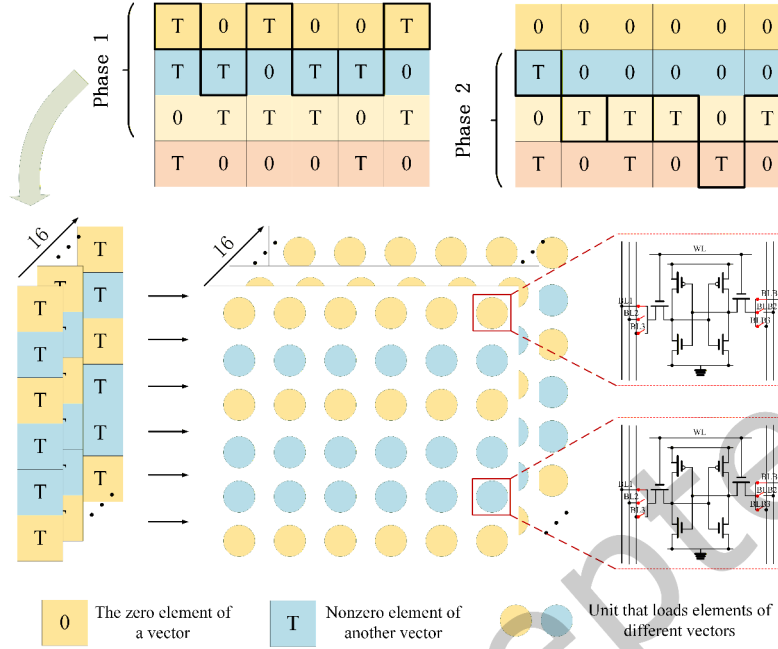


Fig. 8. Co-Optimization Strategy for Sparse Matrix Multiplication

mapping of data on SRAM-PIM, zero elements of input sparse vectors can also be optionally skipped during loading. In this case, rows that should be loaded with zero elements can continue to load non-zero elements from other sparse vectors, reducing the waste of resources. The calculated results of different sparse vectors can be accumulated to the corresponding partial sum through the index to obtain the accurate output result.

Implementing this scheme requires lightweight preprocessing before data loading: the scheduling module determines which elements from each sparse vector should be loaded in a given stage. This can be implemented using existing structures (input buffers, index buffers, queue buffers, and a controller), without adding dedicated new components. Non-zero values and their indices are written into an input buffer and an index buffer in advance; the input buffer stores values, while the index buffer stores (row_{idx}, col_{idx}) .

Before computation, the controller selects first L sparse vectors in the buffer as candidates. Then chooses non-zero entries to maximize column coverage within the current stage, and labels them according to their column index. The controller keeps choosing until either all column indices are covered at least once or the traversal of L candidates has been completed. On the one hand, it ensures that each column of the array has input data to participate in the calculation at this stage, so that the array cells are not idle. On the other hand, by limiting the search range to L , we can avoid wasting scheduling time by over-searching in the case of uneven sparsity distribution.

Subsequently, the controller performs an input-merging step. Specifically, the 256 wordlines (WLs) of the array correspond one-to-one to the column indices of the input vector. Accordingly, the controller constructs, for each array, a stage input queue of length 256 and initializes all entries to zero. For each selected non-zero entry $(row_{idx}, col_{idx}, value)$, the controller places it into the corresponding position of the queue according to the scheduling decisions. After the queue is filled, the controller pushes the data to the wordline drivers/input DACs, thereby loading 256 input values into the 256 array rows in a single stage launch. In addition, each array row is associated with an independent BL/BLB circuit configuration. When $L=3$, a 3-bit setting “100” indicates

that the switch for BL/BLB1 on that row is enabled, while the other two are disabled. This configuration ensures that only the target bitline pair participates in conduction in each operation, preventing current-path crosstalk and enabling row-wise results to be accumulated/aggregated as intended, thus producing correct outputs. The BL/BLB configuration is derived from the index information of the merged vector after the search: the position of an element's row index within the candidate window L determines its setting. For example, if the element belongs to the second vector among the L searched vectors, the row is configured as "010", meaning BL/BLB2 is enabled and the others are disabled. As a special case, if the element is zero, the configuration is set to "000", indicating that all switches are turned off. Subsequently, the array element performs numerical multiplication and accumulates the results into the corresponding partial sums, completing one full computation phase.

As shown in Figure 8, with the setting $L=3$, the controller selects the non-zero elements from the first 3 rows of the buffer in the first phase: The first row contains (1,1), (1,3), and (1,6), and the second row contains (2,2), (2,4), and (2,5). At this point, the column indexes 1-6 have been completely covered, so the selection ends and the computation proceeds. When the computation is complete, line 1 is moved out of the buffer because there are no remaining elements. In the second phase, the controller continues to retrieve the unloaded non-zero elements in rows 2, 3 and 4, where row 2 contains (2,1), row 3 contains (3,2), (3,3), (3,4), (3,6), and row 4 contains (4,5), again achieving coverage of all column indices. Through this "row first + column coverage" strategy, each stage can ensure that the array elements are fully utilized, while trying to limit the overhead of preprocessing to a low level, which will not significantly affect the overall computational efficiency.

The original SRAM analog circuit array is not suitable for data sparsity processing due to the array size and simple interconnection. In order to support the calculation process of sparse matrix multiplication in transformer, additional circuits need to be added to the SRAM array so that it can load as many sparse vectors as possible, and ensure that it can get correct output results while making full use of the SRAM units on the array. As shown in Figure 8, additional BLs and BLBs are added to each array row, and the circuit switches on the circuits connected to the SRAM cells are used to select the current generated by the SRAM cells and then accumulate it on each BL/BLB circuit. Therefore, multiple array circuits that perform the same matrix operation simultaneously can be configured according to the sparsity and scheduling scheme of the S-vector. In the above example, the data loaded for the first time is the non-zero element corresponding to column index (1,3,6) of the first row and the non-zero element corresponding to column index (2,4,5) of the second row. Then, when the array is configured, the switch corresponding to BL1/BLB1 of row (1,3,6) will be turned on. The switches corresponding to BL2/BLB2 of row (2,4,5) will be turned on and all other BL switches will be turned off, ensuring that the elements of each sparse vector are properly computed and accumulated.

In terms of overhead, the additional overhead mainly includes the memory access overhead and search overhead caused by the index. Each non-zero element needs to be stored (row_{idx}, col_{idx}), which increases the storage and bandwidth requirements. The search scheduling overhead is related to the size of L and the sparsity of matrix, and the controller needs to scan L sparse vectors in the buffer. When L is large, the resource utilization is high, while L is small, the search overhead is reduced.

The proposed mechanism introduces an extra per-stage overhead of $O(LN)$, leading to an overall additional time complexity of approximately $O(nnz \cdot L)$, where $nnz \ll N^2$ denotes the number of non-zero elements. Consequently, the total overhead is small compared to the main computation cost $O(N^2M)$.

5 EVALUATION

In this section, we evaluate the effectiveness of various optimization methods of HyPIM in accelerating transformer-based LLM inference. Firstly, we introduce the impact of approximation algorithms on workload accuracy in HyPIM, as well as the analysis of area and power consumption. Then we conduct the overall performance, ablation, and real-time analysis to clarify the performance impact of various optimization schemes on HyPIM.

Finally, we conduct scalability experiments to analyze the impact of increasing the number of compute units and the number of tokens on the overall performance.

Table 2. Configuration Parameters of HyPIM

Components	Parameter	Value
ReRAM	Number of ReRAM layer	4
	Number of Tile of per ReRAM layer	16
	Number of array of per Tile	16
	Number of subarray of per array	16
	Size of subarray	256×256
	t_{set}/t_{reset}	10 ns
SRAM	Number of SRAM layer	2
	Number of Tile of per SRAM layer	16
	Number of array of per Tile	1
	Number of subarray1 of per array	8
	Size of subarray1	256×256
	Number of subarray2 of per array	4
	Size of subarray2	256×128
	Number of subarray3 of per array	4
	Size of subarray3	256×64
Interconnection	The write recovery time t_{SWR}	0.163 ns
	Delay of Horizontal/interlayer	219/16 ps
	Average Hop Count of 2D/3D	5.33/3.81
	MIVs	9.3×10^6
	DAC	1-bit
	ADC	8-bits

In the experiment, the 3D hybrid PIM architecture is defined as H3, the approximation algorithm for attention is defined as A1, and the sparse data processing for attention is defined as A2.

Parameter: The simulation parameters for the ReRAM, including tile configuration, area, and power, are derived from prior research [43]. Its simulation is based on the Verilog-A model [5] for ReRAM. Additionally, the SRAM simulation parameters are based on information from [12]. In the process of architecture modeling, we uniformly use 45nm process library for synthesis and simulation. The digital signal circuit is implemented based on Verilog HDL, and the area and power consumption are synthesized by Synopsys Design Compiler (DC). The analog signal circuit is implemented by SPICE, and the area and power consumption are simulated by Virtuoso. For storage, CACTI [39] was used for SRAM and NVSIM [11] was used for ReRAM. The stacking effect of 3D ReRAM and SRAM is modeled by Destiny-3D [41]. The main network part is simulated based on DSENT [48]. The key design parameters of HyPIM are shown in Table 2. In our experiments, L is set to 4.

Baseline: The following architecture or hardware is used as the baseline (1) Nvidia Tesla V100. (2) Newton [17]: a DRAM-based PIM architecture. (3) TransPIM [54]: a DRAM-based PIM architecture for transformer. (4) HAIMA [10]: A Hybrid SRAM and DRAM PIM architecture for transformer. (5) HARDSEA [34]: A Hybrid SRAM and ReRAM PIM architecture for transformer. (6) H3DAtten [30]: A Hybrid SRAM and ReRAM PIM architecture for transformer.

Benchmark: We used four representative transformer-based LLMs to evaluate task execution efficiency on HyPIM. These are facebook’s BERT-base and BERT-large models and Google’s BART-base and BART-large models. All of them perform the multi-sentence classification inference task on the MNLI dataset. This task requires the model to capture the semantic association based on the premise and hypothesis sentence through the attention mechanism, and classify the relationship between them as "entailment", "contradiction" or "neutral". In order to ensure the fairness and consistency of the results, the maximum length of the input sequence is set to 512 tokens, which effectively controls the computational complexity and ensures that the performance of different scale models in the same hardware environment can be truly reflected.

Evaluation Metrics: In the experimental evaluation, we design indicators from five aspects to fully reflect the efficiency and cost of HyPIM in executing transformer tasks: (1) Area and power consumption: The hardware resource occupation and energy consumption level are evaluated by area (mm^2) and power (mW), reflecting the energy efficiency ratio and integration degree; (2) Accuracy: Based on the classification task of MNLI dataset, the standard accuracy index is used and compared with GPU to verify the reliability of HyPIM in maintaining semantic reasoning ability. (3) Performance: Based on the GPU running cycle, the results are normalized to fairly show the performance of HyPIM in execution speed and throughput. (4) Real-time performance: delay and throughput are used to measure the response efficiency and processing ability of the system under different loads, reflecting the real-time processing potential; (5) Resource utilization: array utilization and cell utilization are investigated to reflect the overall parallelism and cell utilization efficiency respectively, and the advantages and potential bottlenecks of resource scheduling are revealed to provide a basis for subsequent optimization.

Table 3. Area and Power of PiM Architecture

	Size	Area (mm ²)	Power (mW)	Specification	Other PIM Architecture
R-MVMU	256×256×8b	0.0043	15.36	/	Newton: Total Area: 88.3695mm ² Total Power: 67.82W TransPIM: Total Area: 83.0828mm ² Total Power: 69.44W HAIMA: Total Area: 98.9763mm ² Total Power: 62.35W HARDSEA: Total Area: 26.5231mm ² Total Power: 50.59W H3DAtten: total Area: 19.5598mm ² Total Power: 65.03W
S-MVMU	256×64×8b	0.0388	23.03	Original	
	256×128×8b	0.0775	30.16		
S-MVMU-A2	256×256×8b	0.1550	79.92	Extra BL and BLB	
	256×64×8b	0.0720	39.04		
	256×128×8b	0.1441	42.88		
	256×256×8b	0.2881	118.16		
DAC	1bits	0.0185	5.03	1 bit; 8×256/Unit	
ADC	8bits	0.02792	41.86	8 bit; 8/Unit	
S&A	/	0.0016	0.29	256/Unit	
ACC	/	0.0033	1.03	256/Unit	
Active Unit	8KB LUT	0.0248	213.99	/	
Global buffer	64KB	0.3091	526.42	/	
Index buffer	2KB	0.0427	175.10	/	
Interconnection	MIV+NoC	5.122	8.77k	/	
Others	/	1.5419	11.37k	/	
HyPIM Total	/	16.474	58.88k	/	

5.1 Area and Power Evaluation

To ensure a fair comparison, we re-implemented all candidate architectures using a 45-nm technology library in DC. We normalized the total area and power of each architecture to the level required to execute tasks of the same order of magnitude within a unit time, enabling an end-to-end evaluation. The area and power statistics are summarized in Table 3. As shown in the table, HyPIM achieves the smallest total area among all designs: it reduces area by 15.78% compared with the second-best baseline, H3DAtten, and by more than 80% relative to the remaining alternatives. In terms of power, HyPIM is only slightly higher than HARDSEA, yet still remains in the top tier. Furthermore, when considering the area–power product, HyPIM reduces this metric by 23.74% and 27.71% compared with H3DAtten and HARDSEA, respectively, indicating a superior balance between area and power and demonstrating a clear advantage.

Table 3 shows the area and power performance of R-MVMU, S-MVMU and its improved version S-MVMU-A2 in HyPIM under different sizes, and makes comprehensive statistics of peripheral modules (DAC, ADC, buffer unit, etc.) in HyPIM. It can be seen from the results in the Table 3 that R-MVMU maintains a low level in both area and power consumption. We default the 256×256 SRAM array to the original array. As shown in Table 3, the SRAM array area and energy consumption of 256×128 and 256×64 set in A1 are significantly smaller than those of the original array. Therefore, setting arrays of different sizes to handle different tasks can effectively improve the hardware performance. Secondly, in A2, an array with additional BL and BLB lines was set up, which increases

the area overhead by about 85% and the power overhead by 42-68% compared with the original array of the same size in terms of area. This is not ideal, but if the array can effectively deal with sparse data, so as to improve the utilization of resources, it seems not unacceptable. In addition, although the additional index incurs an area overhead of 0.0427 and a power overhead of 175.10, the overhead incurred by the index is still small because the index is shared by multiple cells with an index buffer. Analog circuit array requires additional ADC and DAC modules to perform the computation, which will bring huge area and energy overhead. When the array needs to output high-precision data, the area and power consumption of the ADC and DAC modules will exceed the area and power consumption of the actual PIM analog crossbar array, which offsets the area and power advantages of the analog circuit PIM and 3D stacking technology to some extent.

5.2 Accuracy Evaluation

In order to analyze the influence of early termination algorithm and sparse processing method on the accuracy of the final result. The approximation algorithm is integrated into the official open source implementation of google-BERT and Facebook. Acc, the main accuracy metric in MNLI's relevant papers, was used for evaluation. All four workloads are implemented in pytorch, and all are re-trained with multiple rounds of fine-tuning downstream tasks on top of the pre-trained model.

Impact of judge start time. The Figure 9(a) illustrates the impact of the time of early termination of the start of the algorithm during the attention score matrix calculation on the final accuracy for the four workloads. The specific scheme is to change the value of R to check how to affect the classification accuracy, such as selecting $\frac{n}{4}$, $\frac{n}{2}$, and $\frac{3n}{4}$ bits in the n -bit bit serial to end the calculation and begin to judge.

As shown in the Figure 9(a), starting judge later has less impact on the accuracy of the final result, and as judge starts earlier, the accuracy will be lower and lower. When the judgment starts at the early stage of the calculation process, the subsequent calculation can still affect the size of the final calculation result to a large extent. Therefore, discarding part of the smaller values at this stage may discard the larger values of the actual final result, thus affecting the accuracy of the final classification. However, when the judgment begins at the later stage of the calculation process, the subsequent calculation results are basically small, and the impact on the final calculation results is also small. At this time, the final calculation result corresponding to the smaller value of the judgment is also basically small, so it will not affect the final classification accuracy. In the subsequent experiment, $R=\frac{n}{2}$ was selected as the balanced choice to conduct the experiment.

Impact of the judge threshold. The Figure 9(b) shows the influence of the judge threshold on the final classification accuracy of four workloads during the calculation of the attention score matrix. The solution is to check how the approximation algorithm affects the classification accuracy by changing the threshold value, for example, 2%, 5%, 10% of the maximum value. The elements less than the threshold will be discarded. As shown in the Figure 9(b), although the classification accuracy of the model is in a decreasing trend with the increase of the threshold value, the overall accuracy is still quite high. This proves that the partial sum of the computation in the high bits (early stages) basically determines the final accumulation result, and the computation can be terminated early in the earlier computation process. However, the choice of threshold should not be too aggressive, and the influence of too large threshold on classification accuracy can not be ignored. In the following experiment, $T=5\%$ will be selected as the balanced choice.

The above experiments demonstrate the effectiveness of the approximation algorithm. The experimental results show that approximation algorithms can be roughly divided into aggressive approximation algorithms and conservative approximation algorithms according to the choice of starting judge time M and judgment threshold T . The aggressive approximation algorithm may discard more elements with higher scores due to the inaccurate judgment of the results of each row, resulting in a lower final classification accuracy. However, since it reduces more computation, the relative execution efficiency will be higher. On the contrary, selecting

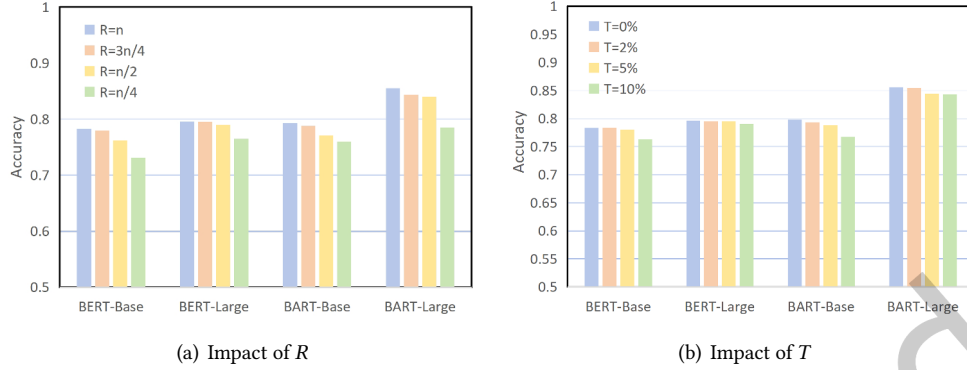


Fig. 9. Evaluation Results of Accuracy

the conservative strategy has almost no impact on the accuracy of the model, but the execution efficiency is relatively low. Therefore, different scenarios need to choose different setup schemes.

Aggressive approximation algorithms are more suitable for scenarios with high efficiency requirements and relatively low accuracy requirements. However, conservative approximation algorithms are more suitable for scenarios with high accuracy requirements and relatively low efficiency requirements.

In RERAM and SRAM integrated memory and computing, the non-ideality of the device will bring noise and bias, which will affect the accuracy of the operation. The common problems of RERAM include conductance randomness, finite precision, drift and IR-drop. SRAM may suffer from threshold mismatch, read and write failures, and bit flipping. These errors will accumulate layer by layer in the matrix calculation, leading to a decrease in model accuracy. In general, the accuracy will be significantly reduced when the effective bitwidth is insufficient or the noise is too large, but the impact can be effectively mitigated by noise-aware training, calibration, or redundant design.

5.3 Performance Evaluation

For an overall performance evaluation of HyPIM, we implemented a cycle-level simulator and integrated it into the open-source implementations of the four workloads described above. We normalize the performance of all architectures with respect to that obtained using Nvidia Tesla V100.

Performance analysis. Figure 10(a) shows the speedup of different architectures when performing transformer inference compared to the baseline. Compared to HAIMA, TransPIM, our architecture achieves a $1.53\times$ - $1.76\times$ improvement in inference latency. Due to the hardware characteristics of DRAM, we can only implement near-memory computation on DRAM. SRAM and ReRAM are not identical, they can integrate the computation unit closer to the memory and decrease the frequency of read/write operations. However, they have their own limitations that render them unsuitable for certain operations. Therefore, for complex network structure, hybrid PIM architecture is a better choice in the future. At the same time, the optimization scheme for the attention mechanism which accounts for a large amount of calculation in the PIM architecture reduces the calculation amount of the two core operations in the mechanism, which is also a non-negligible part of the performance improvement in HyPIM. The figure shows that aggressive approximation algorithms for attention lead to more performance gains. The performance improvement brought by each component will be analyzed in ablation evaluation. The performance comparison between HyPIM-2D and HyPIM-3D shows that 3D stacking not only saves more area, but also significantly improves the performance, which reflects its advantages in memory and computing integration. Finally, the performance comparison between HyPIM-3D and HyPIM shows that

incorporating an SW-NoC into the 3D architecture provides stable gains over the 3D design without it, with a performance improvement of approximately $1.08\times$ – $1.12\times$. Under our workloads, where communication is relatively balanced, the achieved benefit is therefore limited.

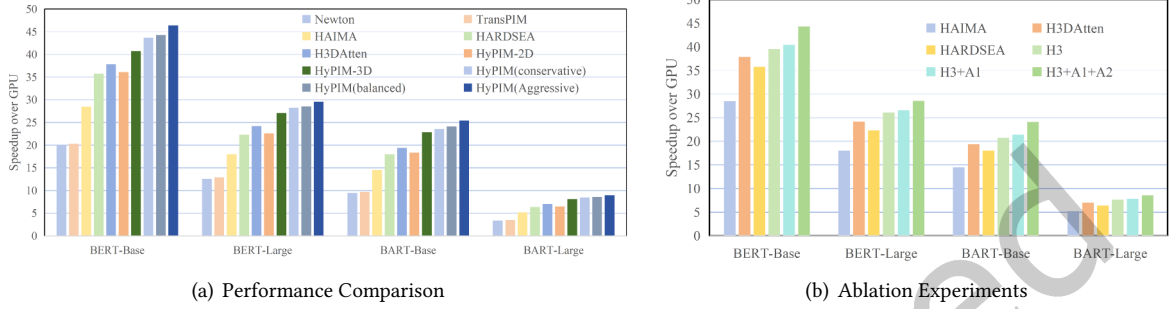


Fig. 10. Performance Comparison and Ablation Experiments. HyPIM-2D: a non-3D-stacked tiled architecture interconnected via a 2D mesh. HyPIM-3D: a 3D-stacked architecture interconnected via a 3D mesh. HyPIM: extends HyPIM-3D by additionally incorporating an SW-NoC; other configurations remain essentially the same.

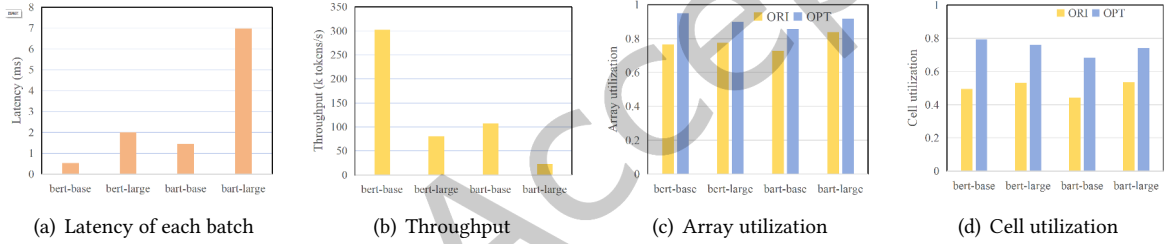


Fig. 11. Real-time and resource utilization analysis. ORI: the baseline implementation that does not include the Mixed Granularity Parallel Strategy, Early Calculation Termination Strategy and Co-Optimization Strategy for Sparse Matrix Multiplication; OPT: the integrated implementation with all proposed strategies enabled.

Real-time analysis. We test the processing latency and throughput performance of the four models on HyPIM. In the experiment, the input sequence length of the BERT model is 162, and the input sequence length of the BART model is 156. In each group of experiments, 1000 sequences are processed to obtain their average value. As shown in Figure 11(a) and Figure 11(b), the results show that the total time consumption of BERT-base is 0.536s, the average delay per task is only 0.54ms, and the throughput is up to 302K tokens/s. The total time consumption of BERT-large is 1.998s, with an average of 2ms per task and a throughput of 81K tokens/s. The total time of BART-base is 1.459s, the average time per task is 1.46ms, and the throughput is 107K tokens/s. The total time of BART-large is 6.986 seconds, with an average of 6.99ms per task and a throughput of 22K tokens/s. From the perspective of real-time performance, in this experimental results, the average single-task delay of the four models under 1000 task batches is far less than 10ms, and even the largest BART-large is only 6.99ms/task. This shows that our work has strong real-time processing ability when dealing with large-scale models such as BERT and BART models, and can meet the real-time requirements of low latency in natural language processing scenarios.

Resource utilization analysis. Figure 11(c) shows that in the execution process of the four models BERT-base, BERT-large, BART-base and BART-large, the array utilization using the optimization strategy is generally higher

than that of baseline, and the improvement range is between 10% and 20%, and the highest is close to full utilization. This indicates that the optimization strategy can effectively alleviate the problem of array resource waste. Further, it can be seen from Figure 11(d) that the cell utilization rate is improved more obviously after using the optimization strategy. Compared with the bare-metal machine, the improvement range is generally above 20%-30%, and the maximum utilization rate can reach nearly 80%, indicating that the computing units inside the array are more fully utilized.

From the experimental results, the optimization strategy has a significant improvement compared with the original scheme in terms of both array utilization and cell utilization. This improvement mainly comes from the synergy of three optimization strategies. Firstly, the weight replication strategy was introduced to balance the data distribution to a certain extent, so as to improve the efficiency of array mapping. Secondly, the configuration of arrays with different sizes can adapt to the computing requirements and reduce the phenomenon of resource fragmentation. Finally, the hardware and software co-optimization strategy for sparse computing was proposed to further improve the hardware utilization under sparse computing. On the whole, the comprehensive application of these strategies significantly improves the resource utilization efficiency at the array and cell levels, and provides strong support for improving the inference performance of sparse models.

5.4 Ablation Evaluation

To analyze the performance of HyPIM in depth, we study the different components separately to check the performance improvement that each component can provide. We divided the three optimization schemes on HyPIM, redesigned the cycle-level simulator respectively, and integrated them into the open source implementation of the above four workloads. We also normalize the performance of all schemes with respect to that obtained using Nvidia Tesla V100. In addition, we compared the end-to-end performance of the integrated system with HAIMA, HARDSEA, and H3DAtten. As shown by the overall results in the Figure 10(b), even when evaluated against the latest baselines, each key design choice/module in this paper consistently yields explainable performance improvements. This indicates that the efficient parallel dataflow and pipelining strategy provided by our HyPIM architecture can deliver substantial performance gains.

Second, we analyze the performance improvements contributed by each component. As shown in Figure 10(b), the 3D stacked hybrid storage technology can bring $1.38\times$ - $1.47\times$ performance improvement compared to the baseline. This is mainly due to the more efficient and suitable processing of transformer operations by the hybrid storage architecture, and the faster data transmission speed and shorter data transmission path brought by the stacking of memory slices in 3D-PIM architecture.

Then, as depicted in Figure 10(b), the software and hardware collaboration scheme of A1 can bring $1.02\times$ to $1.05\times$ performance improvement. This is mainly due to the reduction of part of the computation and the efficient utilization of hardware resources brought by the software-hardware cooperation scheme of approximation algorithms with early termination. Regular approximation algorithms are not general on PIM architectures due to their data rearrangement as well as preprocessing schemes. The approximation algorithm in our scheme fully considers the rigid interconnection between the nodes of the analog array, does not change the data layout of the weights, and designs early termination in the calculation process of the bit-serial calculation mode. The experimental results prove the effectiveness of our scheme on the PIM architecture.

Finally, as can be seen in Figure 10(b), the software and hardware collaboration scheme of A2 can bring $1.09\times$ to $1.13\times$ performance improvement. This is mainly due to the efficient processing of sparse data on the PIM architecture and the efficient utilization of hardware resources. By adjusting the array, the array can handle multiple sparse vectors at the same time, thus improving the resource utilization and efficiency. It is worth noting that the source of the sparse data is the output of the operations in A1, which means that the quality of the approximation algorithm in A1 can affect the quality of the optimization scheme in A2.

5.5 Scalability Evaluation

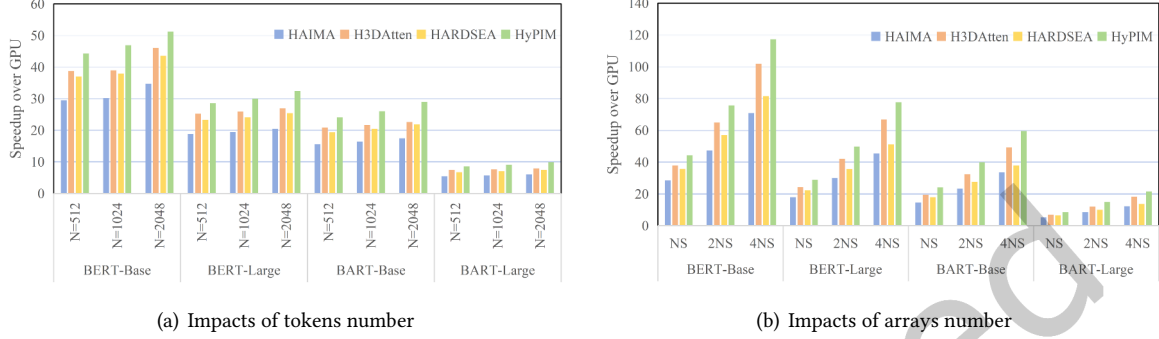


Fig. 12. Scalability Analysis

Since the sequence data length is variable, the speedup of HyPIM architecture may vary with different sequence lengths. To verify the performance improvement of HyPIM for variable-length tasks, the model is extended to support inference tasks with longer sequences. Figure 12(a) compares the performance improvement of the HyPIM architecture for sequence lengths of 512, 1024, and 2048, and also includes comparisons with HAIMA, HARDSEA, and H3DAaten. First, the results indicate that, even as the sequence length varies, HyPIM consistently achieves a $1.11\times$ – $1.66\times$ performance improvement over these state-of-the-art architectures, demonstrating strong robustness. The results show that the speedup ratio of HyPIM architecture increases as the sequence length increases. The main reason for this is the optimization of the attention mechanism. When the sequence length is short, the proportion of attention mechanism calculation is relatively small, and the A1 and A2 acceleration only have a small improvement. For long sequence data, the calculation proportion of attention mechanism is relatively high, the A1 and A2 acceleration effect is significant, and the overall efficiency of HyPIM is significantly improved. Therefore, the HyPIM architecture with attention optimization mechanism is more suitable for processing long sequence data.

Then we conducted experiments to evaluate the architectural scalability. Figure 12(b) presents a comparison of the acceleration achieved by each architecture under different numbers of arrays, including HAIMA, HARDSEA, and H3DAaten. Here, NS denotes the standard number of compute arrays in the architecture. The experimental results show that all architectures benefit substantially as the number of arrays increases, while HyPIM consistently outperforms the others. HyPIM also exhibits strong stability, still delivering a $1.15\times$ – $1.75\times$ performance improvement over these baselines. HyPIM can copy multiple weights to other arrays in parallel when the resources are sufficient, and the overall calculation adopts pipeline architecture, HyPIM can still effectively use computing resources and achieve significant performance improvement when the computing array is proportional to the increase.

6 Related work

Traditional NN Accelerators: The increasing size of neural networks leads to longer inference delays, so a variety of NN accelerators [1, 40, 53] have now emerged. A large number of customized CNN accelerators have been proposed to accelerate CNN inference and training by exploiting the sparsity of weights or activations and the high parallelism in execution. SCNN [40] achieves model compression and acceleration by reducing unnecessary connections and computations. SnapeA [1] designs a predictable early activation strategy to determine part of

the output data to zero in advance based on the Relu feature. In this paper, we focus on transformers based on attention mechanism.

Transformer Accelerator: Due to the high calculation proportion of attention mechanism in the execution process, some accelerators [7, 13–16, 26, 37, 42] have been proposed for attention mechanism. A^3 [15] applies the approximation algorithm to the attention mechanism through preprocessing sorting, which reduces the computational overhead by terminating part of the computation early. ELSA [16] improves the approximation method by concisely representing the estimated angles to calculate the similarity between them, filtering out the weights with less calculation. However, the above works basically only focus on the attention score calculation in the attention mechanism. Dota [42] filters the weights by generating a mask matrix of score weights through low-rank transformation of the matrix, and designs an end-to-end inference acceleration by using the parallelism between tokens. Sanger [37] focuses on the attention score calculation and attention output calculation of the attention mechanism at the same time, and uses the quantitative prediction based on the attention matrix to dynamically sparse attention. Some works [13] focus on both the FFN and attention modules and optimizes them jointly. Our paper chooses to focus on both the attention score calculation and the attention output calculation of the attention mechanism simultaneously and implements transformer acceleration through the software and hardware co-design of the early termination and sparse processing strategy.

PIM Accelerator. In order to reduce the cost of data movement during task execution, various PIM accelerators have been proposed. Newton [17], and TransPIM [54] are digital PIM accelerators designed based on DRAM. They adopt near-memory architecture which places the computing units in DRAM bank to reduce overhead by high-speed in-memory bandwidth in HBM. In contrast, TWT [47], Colonnade [24] are SRAM-based CIM architectures [27, 52], in which part of the work adopts digital CIM arrays to integrate the computing cells near the storage cells. In the other part of the work, the analog PIM array is used to complete the analog signal calculation in the memory cell by using the characteristics of the SRAM cell. ReRAM, a new non-volatile memory technology, comes into the field of PIM design because of its high storage density and high efficiency [46]. Some PIM architectures such as ISAAC [43], W2W-PIM [31], PRIME [6], and RENO [35] use ReRAM crossbar memory arrays to perform analog data computations. Based on this, the existing research has explored many aspects of large model acceleration under the framework of PIM. Accelerators based on 3D stacked DRAM [44] provide the large capacity required for large models. PipePIM [19] improves the utilization of DRAM-based PIM architecture with pipeline and double buffering. TransPIM [54] adopts a tokens-based data flow and makes lightweight modifications in the conventional High-Bandwidth Memory (HBM) architecture to support transformer's hybrid PIM/NMC processing and efficient data communication. However, they do not fully consider the characteristics of different modules, and are still limited to a single hardware PIM architecture. By taking into account the characteristics of different computations in transformers [21], HAIMA [10] proposes hybrid PIM architectures that exploit the cooperation between SRAM and DRAM to accelerate different operations. SpecPIM [29] uses heterogeneous PIM architecture to accelerate speculative reasoning, and integrates PIM engine in GDDR6. H3d-transformer [38] uses 3D technology to heterogeneous DRAM-PIM and TPU for different module characteristics of Transformer. H3DAtten [30] proposed the heterogeneous 3D-PIM architecture of ReRAM+SRAM and considered the hardware characteristics. However, the above works basically do not consider the combination of PIM architecture to optimize the attention module with large computation in the transformer model. HARDSEA [34] also uses ReRAM+SRAM heterogeneous architecture and optimizes attention processing, but lacks 3D stacking support and is limited in area efficiency. In this paper, Heterogeneous 3D reraam and sram architecture is used to accelerate different operations of transformer. At the same time, software and hardware co-optimization is done for the attention module with a large calculation, so as to realize the double acceleration of data movement and computation cost.

7 Conclusion

In this paper, we propose HyPIM, a 3D hybrid memory PIM architecture that employs optimization strategies for transformers and attention modules to accelerate LLMs. The architecture leverages high-efficiency SRAM storage and high-capacity ReRAM storage co-processing to address the imbalance between storage and computing resources that both traditional architectures and conventional PIM architectures face when processing transformer modules. Additionally, we recognize the importance of the attention module in transformer-based LLM reasoning and reduce its impact on overall reasoning delay through a software and hardware co-design of an early termination algorithm and a sparse data processing algorithm for the two core calculations of the attention module. Our architecture achieves up to a $1.27\times$ – $1.67\times$ latency improvement over other state-of-the-art PIM architectures, such as Newton, TransPIM, HAIMA, HARDSEA, and H3DAtten, while also demonstrating strong robustness and scalability, thereby opening up opportunities for further advances in LLM development.

Our work focuses on real-time LLM workloads, specifically large model applications used in embedded and real-time systems. Additionally, our approach accelerates LLMs with limited resources, improves the real-time response speed of large model applications like LLMs, and efficiently executes applications such as embedded end-side LLMs. At the same time, it is compatible with existing mainstream processors, making it easy to integrate into various embedded platforms.

Accelerating LLMs through software and hardware collaboration holds great promise for the future of embedded real-time workloads. By leveraging hardware accelerators and advanced hardware-software combined algorithms, embedded devices can perform complex language model inference with low power consumption, low latency, and high efficiency. This provides strong support for application scenarios such as edge computing, real-time response, and multitasking. It will accelerate technological innovation in fields such as autonomous driving and intelligent healthcare, and promote the widespread adoption of intelligent embedded systems.

There is still significant room for improvement in our work. By further customizing the design of hardware accelerators and adapting LLMs, as well as optimizing intelligent algorithms to align with architectural characteristics, we can achieve greater efficiency and stronger scalability.

In short, this architecture provides an efficient and economical solution for LLM real-time workloads, which can be further integrated with embedded real-time systems to serve various practical application scenarios more intelligently, efficiently, and quickly.

References

- [1] Vahideh Akhlaghi, Amir Yazdanbakhsh, Kambiz Samadi, Rajesh K. Gupta, and Hadi Esmaeilzadeh. 2018. SnaPEA: Predictive Early Activation for Reducing Computation in Deep Convolutional Neural Networks. In *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*. 662–673.
- [2] Aayush Ankit, Izzat El Hajj, Sai Rahul Chalamalasetti, Geoffrey Ndu, Martin Foltin, R Stanley Williams, Paolo Faraboschi, Wen-mei W Hwu, John Paul Strachan, Kaushik Roy, et al. 2019. PUMA: A programmable ultra-efficient memristor-based accelerator for machine learning inference. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*. 715–731.
- [3] Soroush Bateni and Cong Liu. 2018. Apnet: Approximation-aware real-time neural network. In *2018 IEEE Real-Time Systems Symposium (RTSS)*. IEEE, 67–79.
- [4] Kyungwook Chang, Deepak Kadedotad, Yu Cao, Jae-sun Seo, and Sung Kyu Lim. 2017. Monolithic 3D IC designs for low-power deep neural networks targeting speech recognition. In *2017 IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED)*. IEEE, 1–6.
- [5] Pai-Yu Chen and Shimeng Yu. 2015. Compact modeling of RRAM devices and its applications in 1T1R and 1S1R array design. *IEEE Transactions on Electron Devices* 62, 12 (2015), 4022–4028.
- [6] Ping Chi, Shuangchen Li, Cong Xu, Tao Zhang, Jishen Zhao, Yongpan Liu, Yu Wang, and Yuan Xie. 2016. PRIME: A Novel Processing-in-Memory Architecture for Neural Network Computation in ReRAM-Based Main Memory. In *43rd ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2016, Seoul, South Korea, June 18–22, 2016*. IEEE Computer Society, 27–39.

- [7] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. 2019. Generating Long Sequences with Sparse Transformers. *CoRR* abs/1904.10509 (2019). arXiv:1904.10509
- [8] Sourav Das, Dongjin Lee, and Partha Pratim Pande. 2015. Small-world network enabled energy efficient and robust 3D NoC architectures. In *Proceedings of the 25th edition on Great Lakes Symposium on VLSI*. 133–138.
- [9] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805* (2018).
- [10] Yan Ding, Chubo Liu, Mingxing Duan, Wanli Chang, Keqin Li, and Kenli Li. 2023. HAIMA: A Hybrid SRAM and DRAM Accelerator-in-Memory Architecture for Transformer. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–6.
- [11] Xiangyu Dong, Cong Xu, Yuan Xie, and Norman P Jouppi. 2012. Nvsim: A circuit-level performance, energy, and area model for emerging nonvolatile memory. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 31, 7 (2012), 994–1007.
- [12] Charles Eckert, Xiaowei Wang, Jingcheng Wang, Arun Subramaniyan, Ravi Iyer, Dennis Sylvester, David Blaauw, and Reetuparna Das. 2018. Neural cache: Bit-serial in-cache acceleration of deep neural networks. In *2018 ACM/IEEE 45th annual international symposium on computer architecture (ISCA)*. IEEE, 383–396.
- [13] Hongxiang Fan, Thomas Chau, Stylianos I. Venieris, Royson Lee, Alexandros Kouris, Wayne Luk, Nicholas D. Lane, and Mohamed S. Abdelfattah. 2022. Adaptable Butterfly Accelerator for Attention-based NNs via Hardware and Algorithm Co-design. In *55th IEEE/ACM International Symposium on Microarchitecture, MICRO 2022, Chicago, IL, USA, October 1-5, 2022*. IEEE, 599–615.
- [14] Jiarui Fang, Yang Yu, Chengduo Zhao, and Jie Zhou. 2021. TurboTransformers: an efficient GPU serving system for transformer models. In *PPoPP '21: 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, Virtual Event, Republic of Korea, February 27- March 3, 2021*, Jaejin Lee and Erez Petrank (Eds.). ACM, 389–402.
- [15] Tae Jun Ham, Sungjun Jung, Seonghak Kim, Young H. Oh, Yeonhong Park, Yoonho Song, Jung-Hun Park, Sanghee Lee, Kyoung Park, Jae W. Lee, and Deog-Kyoon Jeong. 2020. A³: Accelerating Attention Mechanisms in Neural Networks with Approximation. In *IEEE International Symposium on High Performance Computer Architecture, HPCA 2020, San Diego, CA, USA, February 22-26, 2020*. IEEE, 328–341.
- [16] Tae Jun Ham, Yejin Lee, Seong Hoon Seo, Soosung Kim, Hyunji Choi, Sung Jun Jung, and Jae W. Lee. 2021. ELSA: Hardware-Software Co-design for Efficient, Lightweight Self-Attention Mechanism in Neural Networks. In *48th ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2021, Virtual Event / Valencia, Spain, June 14-18, 2021*. IEEE, 692–705.
- [17] Mingxuan He, Choungki Song, Ilkon Kim, Chunseok Jeong, Seho Kim, Il Park, Mithuna Thottethodi, and T. N. Vijaykumar. 2020. Newton: A DRAM-maker's Accelerator-in-Memory (AiM) Architecture for Machine Learning. In *53rd Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2020, Athens, Greece, October 17-21, 2020*. IEEE, 372–385.
- [18] Joo Seong Jeong, Jingyu Lee, Donghyun Kim, Changmin Jeon, Changjin Jeong, Youngki Lee, and Byung-Gon Chun. 2022. Band: coordinated multi-dnn inference on heterogeneous mobile processors. In *Proceedings of the 20th Annual International Conference on Mobile Systems, Applications and Services*. 235–247.
- [19] Taeyang Jeong and Eui-Young Chung. 2024. Pipepim: Maximizing computing unit utilization in ml-oriented digital pim by pipelining and dual buffering. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 43, 12 (2024), 4585–4598.
- [20] Mingoo Ji, Saehanseul Yi, Changjin Koo, Sol Ahn, Dongjoo Seo, Nikil Dutt, and Jong-Chan Kim. 2022. Demand layering for real-time DNN inference with minimized memory usage. In *2022 IEEE Real-Time Systems Symposium (RTSS)*. IEEE, 291–304.
- [21] Shengyi Ji, Chubo Liu, Yan Ding, Qing Liao, and Zhuo Tang. 2024. A Real-time Execution System of Multimodal Transformer through PIM-GPU Collaboration. In *Proceedings of the 61st ACM/IEEE Design Automation Conference*. 1–6.
- [22] Biresh Kumar Joardar, Janardhan Rao Doppa, Partha Pratim Pande, Hai Li, and Krishnendu Chakrabarty. 2020. AccuReD: High accuracy training of CNNs on ReRAM/GPU heterogeneous 3-D architecture. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 40, 5 (2020), 971–984.
- [23] Duckhwan Kim, Jaeha Kung, Sek Chai, Sudhakar Yalamanchili, and Saibal Mukhopadhyay. 2016. Neurocube: A programmable digital neuromorphic architecture with high-density 3D memory. *ACM SIGARCH Computer Architecture News* 44, 3 (2016), 380–392.
- [24] Hyunjoon Kim, Taegeun Yoo, Tony Tae-Hyoung Kim, and Bongjin Kim. 2021. Colonnade: A Reconfigurable SRAM-Based Digital Bit-Serial Compute-In-Memory Macro for Processing Neural Networks. *IEEE J. Solid State Circuits* 56, 7 (2021), 2221–2233.
- [25] Joo-Young Kim, Bongjin Kim, and Tony Tae-Hyoung Kim. 2023. *Processing-in-memory for AI: From Circuits to Systems*. Springer.
- [26] François Lagunas, Ella Charlaix, Victor Sanh, and Alexander M. Rush. 2021. Block Pruning For Faster Transformers. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, EMNLP 2021, Virtual Event / Punta Cana, Dominican Republic, 7-11 November, 2021*, Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (Eds.). Association for Computational Linguistics, 10619–10629.
- [27] Marzieh Lenjani, Alif Ahmed, Mircea Stan, and Kevin Skadron. 2022. Gearbox: a case for supporting accumulation dispatching and hybrid partitioning in PIM-based accelerators. In *ISCA '22: The 49th Annual International Symposium on Computer Architecture, New York, New York, USA, June 18 - 22, 2022*, Valentina Salapura, Mohamed Zahran, Fred Chong, and Lingjia Tang (Eds.). ACM, 218–230.

- [28] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Ves Stoyanov, and Luke Zettlemoyer. 2019. Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. *arXiv preprint arXiv:1910.13461* (2019).
- [29] Cong Li, Zhe Zhou, Size Zheng, Jiaxi Zhang, Yun Liang, and Guangyu Sun. 2024. Specpim: Accelerating speculative inference on pim-enabled system via architecture-dataflow co-exploration. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 950–965.
- [30] Wantong Li, Madison Manley, James Read, Ankit Kaul, Muhannad S Bakir, and Shimeng Yu. 2023. H3datten: Heterogeneous 3-d integrated hybrid analog and digital compute-in-memory accelerator for vision transformer self-attention. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 31, 10 (2023), 1592–1602.
- [31] Xingchen Li, Bingzhe Wu, Guangyu Sun, Zhe Zhang, Zhihang Yuan, Runsheng Wang, Ru Huang, Dimin Niu, Hongzhong Zheng, Zhichao Lu, Liang Zhao, Meng-Fan Marvin Chang, Tianchan Guan, and Xin Si. 2022. Enabling High-Quality Uncertainty Quantification in a PIM Designed for Bayesian Neural Network. In *IEEE International Symposium on High-Performance Computer Architecture, HPCA 2022, Seoul, South Korea, April 2-6, 2022*. IEEE, 1043–1055.
- [32] Yufei Li, Zexin Li, Wei Yang, and Cong Liu. 2023. Rt-lm: Uncertainty-aware resource management for real-time inference of language models. *arXiv preprint arXiv:2309.06619* (2023).
- [33] Zexin Li, Yuqun Zhang, Ao Ding, Husheng Zhou, and Cong Liu. 2021. Efficient algorithms for task mapping on heterogeneous CPU/GPU platforms for fast completion time. *Journal of Systems Architecture* 114 (2021), 101936.
- [34] Shiwei Liu, Chen Mu, Hao Jiang, Yunzhengmao Wang, Jinshan Zhang, Feng Lin, Keji Zhou, Qi Liu, and Chixiao Chen. 2023. Hardsea: Hybrid analog-reram clustering and digital-sram in-memory computing accelerator for dynamic sparse self-attention in transformer. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 32, 2 (2023), 269–282.
- [35] Xiaoxiao Liu, Mengjie Mao, Beiye Liu, Hai Li, Yiran Chen, Boxun Li, Yu Wang, Hao Jiang, Mark Barnell, Qing Wu, and Jianhua Joshua Yang. 2015. RENO: a high-efficient reconfigurable neuromorphic computing accelerator design. In *Proceedings of the 52nd Annual Design Automation Conference, San Francisco, CA, USA, June 7-11, 2015*. ACM, 66:1–66:6.
- [36] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. 2021. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF international conference on computer vision*. 10012–10022.
- [37] Liqiang Lu, Yicheng Jin, Hangrui Bi, Zizhang Luo, Peng Li, Tao Wang, and Yun Liang. 2021. Sanger: A Co-Design Framework for Enabling Sparse Attention using Reconfigurable Architecture. In *MICRO '21: 54th Annual IEEE/ACM International Symposium on Microarchitecture, Virtual Event, Greece, October 18-22, 2021*. ACM, 977–991.
- [38] Yandong Luo and Shimeng Yu. 2024. H3d-transformer: A heterogeneous 3d (h3d) computing platform for transformer model acceleration on edge devices. *ACM Transactions on Design Automation of Electronic Systems* 29, 3 (2024), 1–19.
- [39] Naveen Muralimanohar, Rajeev Balasubramanian, and Norm Jouppi. 2007. Optimizing NUCA organizations and wiring alternatives for large caches with CACTI 6.0. In *40th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO 2007)*. IEEE, 3–14.
- [40] Angshuman Parashar, Minsoo Rhu, Anurag Mukkara, Antonio Puglielli, Rangharajan Venkatesan, Brucek Khailany, Joel Emer, Stephen W. Keckler, and William J. Dally. 2017. SCNN: An Accelerator for Compressed-sparse Convolutional Neural Networks. In *Proceedings of the 44th Annual International Symposium on Computer Architecture (Toronto, ON, Canada) (ISCA '17)*. Association for Computing Machinery, New York, NY, USA, 27–40.
- [41] Matt Poremba, Sparsh Mittal, Dong Li, Jeffrey S Vetter, and Yuan Xie. 2015. Destiny: A tool for modeling emerging 3d nvm and edram caches. In *2015 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 1543–1546.
- [42] Zheng Qu, Liu Liu, Fengbin Tu, Zhaodong Chen, Yufei Ding, and Yuan Xie. 2022. Dota: detect and omit weak attentions for scalable transformer acceleration. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 14–26.
- [43] Ali Shafiee, Anirban Nag, Naveen Muralimanohar, Rajeev Balasubramanian, John Paul Strachan, Miao Hu, R Stanley Williams, and Vivek Srikumar. 2016. ISAAC: A convolutional neural network accelerator with in-situ analog arithmetic in crossbars. *ACM SIGARCH Computer Architecture News* 44, 3 (2016), 14–26.
- [44] Janak Sharda, Po-Kai Hsu, and Shimeng Yu. 2024. Accelerator design using 3D stacked capacitorless DRAM for large language models. In *2024 IEEE 6th International Conference on AI Circuits and Systems (AICAS)*. IEEE, 487–491.
- [45] Prachi Shukla, Vasilis F Pavlidis, Emre Salman, and Ayse K Coskun. 2023. Tread-m3d: Temperature-aware dnn accelerators for monolithic 3-d mobile systems. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 42, 12 (2023), 4350–4363.
- [46] Linghao Song, Xuehai Qian, Hai Li, and Yiran Chen. 2017. PipeLayer: A Pipelined ReRAM-Based Accelerator for Deep Learning. In *2017 IEEE International Symposium on High Performance Computer Architecture, HPCA 2017, Austin, TX, USA, February 4-8, 2017*. IEEE Computer Society, 541–552.
- [47] Jian-Wei Su, Xin Si, Yen-Chi Chou, Ting-Wei Chang, Wei-Hsing Huang, Yung-Ning Tu, Ruhui Liu, Pei-Jung Lu, Ta-Wei Liu, Jing-Hong Wang, Yen-Lin Chung, Jin-Sheng Ren, Fu-Chun Chang, Yuan Wu, Hongwu Jiang, Shanshi Huang, Sih-Han Li, Shyh-Shyuan Sheu, Chih-I Wu, Chung-Chuan Lo, Ren-Shuo Liu, Chih-Cheng Hsieh, Kea-Tiong Tang, Shimeng Yu, and Meng-Fan Chang. 2022. Two-Way Transpose Multibit 6T SRAM Computing-in-Memory Macro for Inference-Training AI Edge Chips. *IEEE J. Solid State Circuits* 57, 2

- (2022), 609–624.
- [48] Chen Sun, Chia-Hsin Owen Chen, George Kurian, Lan Wei, Jason Miller, Anant Agarwal, Li-Shiuan Peh, and Vladimir Stojanovic. 2012. DSENT-a tool connecting emerging photonics with electronics for opto-electronic networks-on-chip modeling. In *2012 IEEE/ACM Sixth International Symposium on Networks-on-Chip*. IEEE, 201–210.
 - [49] Madhava Sarma Vemuri and Umamaheswara Rao Tida. 2023. Small Footprint 6T-SRAM Design with MIV-Transistor Utilization in M3D-IC Technology. In *2023 IEEE 41st International Conference on Computer Design (ICCD)*. IEEE, 118–125.
 - [50] Steven Waskito, Kai Jie Leow, Pramuka Medaranga, Tejas Gupta, Shantanu Chakrabarty, Manoj Gulati, and Ambuj Varshney. 2023. Otter: Simplifying Embedded Sensor Data Collection and Analysis using Large Language Models. In *Proceedings of the 29th Annual International Conference on Mobile Computing and Networking*. 1–3.
 - [51] Yecheng Xiang and Hyoseung Kim. 2019. Pipelined data-parallel CPU/GPU scheduling for multi-DNN real-time inference. In *2019 IEEE Real-Time Systems Symposium (RTSS)*. IEEE, 392–405.
 - [52] Liukai Xu, Songyuan Liu, Zhi Li, Dengfeng Wang, Yiming Chen, Yanan Sun, Xueqing Li, Weifeng He, and Shi Xu. 2022. CREAM: computing in ReRAM-assisted energy and area-efficient SRAM for neural network acceleration. In *Proceedings of the 59th ACM/IEEE Design Automation Conference*. 115–120.
 - [53] Jiaqi Zhang, Xiangru Chen, Mingcong Song, and Tao Li. 2019. Eager pruning: Algorithm and architecture support for fast training of deep neural networks. In *2019 ACM/IEEE 46th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 292–303.
 - [54] Minxuan Zhou, Weihong Xu, Jaeyoung Kang, and Tajana Rosing. 2022. Transpim: A memory-based acceleration via software-hardware co-design for transformer. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 1071–1085.

Received 30 December 2024; revised 10 January 2026; accepted 18 May 2026